

**A PROPOSED DESIGN AND IMPLEMENTATION**

**DSN4091-CAPSTONE PROJECT PHASE-I**

*Phase – I Report*

*Submitted by*

**SHREYANSH UTTAM (21BAI10xxx)**

*in partial fulfillment for the award of the degree  
of*

**BACHELOR OF TECHNOLOGY**

*in*

**COMPUTER SCIENCE AND ENGINEERING  
(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)**

**SCHOOL OF COMPUTING SCIENCE AND ENGINEERING**

**VIT BHOPAL UNIVERSITY SEHORE,  
MADHYA PRADESH - 466114**

September 2026

**VIT BHOPAL UNIVERSITY, KOTRIKALAN, SEHORE**  
**MADHYA PRADESH – 466114**

**BONAFIDE CERTIFICATE**

Certified that this project report titled “A PROPOSED DESIGN AND IMPLEMENTATION FOR AI-POWERED SUPPLY CHAIN INTELLIGENCE, RISK PREDICTION & OPTIMIZATION PLATFORM - NEXUS” is the bonafide work of “SHREYANSH UTTAM (21BAI10xxx)” who carried out the project work (DSN4091- Capstone Project Phase-I) under my supervision.

Certified further that to the best of my knowledge the work reported at this time does not form part of any other project/research work based on which a degree or award was conferred on an earlier occasion on this or any other candidate.

**PROGRAM CHAIR**

**Dr Pradeep Kumar Mishra**

Senior Assistant Professor (Gr-2),  
School of Computing Science Engineering and  
Artificial Intelligence  
VIT BHOPAL UNIVERSITY

**PROJECT GUIDE**

**Dr. XXX,**

Assistant Professor (Gr-2) or other designation  
School of Computing Science Engineering and  
Artificial Intelligence  
VIT BHOPAL UNIVERSITY

The DSN4091-Capstone Project Phase-I Viva Voce Examination is held on \_\_\_\_\_

## ACKNOWLEDGEMENT

First and foremost, we would like to thank the Lord Almighty for his presence and immense blessings throughout the project work.

We would like to thank our internal guide **Dr. XX**, for continually guiding and actively participating in our project, and giving valuable suggestions to complete the project works.

We wish to express our heartfelt gratitude to Dr. Pradeep Kumar Mishra, PC-Lead, School of Computing Science Engineering and Artificial Intelligence for much of his valuable support and encouragement in carrying out this work.

We wish to express our heartfelt gratitude to Dr Pon Harshavardhanan, Dean, School of Computing Science Engineering and Artificial Intelligence for much of his valuable support and encouragement in carrying out this work.

We would like to thank all the technical and teaching staff of the School of Computing Science Engineering and Artificial Intelligence, who extended directly or indirectly all support.

Last, but not least, we are deeply indebted to our parents who have been the greatest support while we worked day and night for the project to make it a success.

## LIST OF FIGURES

| FIGURE NO. | TITLE                                                          | PAGE NO. |
|------------|----------------------------------------------------------------|----------|
| 1.0        | System Architectural Design & End-to-End Pipeline              | 14       |
| 2.0        | Closed-Loop Decision Intelligence Workflow                     | 15       |
| 3.0        | Relational Schema and Digital Twin Entity-Relationship Diagram | 16       |
| 4.0        | Executive Overview Control Room Dashboard Interface            | 17       |
| 5.0        | Digital Twin Geospatial Network Map Across Indian Corridors    | 18       |
| 6.0        | Scenario Simulation & Non-Destructive State Shock Control Flow | 22       |
| 7.0        | Machine Learning Demand Forecasting Benchmarks                 | 26       |
| 8.0        | Operational Cost Optimization Waterfall Under Disruption       | 27       |

## ABSTRACT

Modern multi-echelon enterprise supply chains operate under conditions of persistent volatility, characterized by frequent supplier disruptions, transit delays, freight bottlenecks, and sharp demand surges. Conventional Enterprise Resource Planning (ERP) systems and Business Intelligence (BI) dashboards suffer from significant operational latency because they function predominantly as passive, descriptive reporting tools. They record historical failures rather than anticipating risk, diagnosing upstream failure cascades, or mathematically prescribing cost-effective counteractions. Consequently, supply chain managers frequently resort to manual, sub-optimal heuristics that amplify Bullwhip effects, incur massive stockout penalties, and severely compromise end-to-end service levels.

This project addresses these critical industry challenges through the design, implementation, and empirical validation of NEXUS — an AI-powered supply chain intelligence, predictive risk management, and multi-echelon linear optimization platform. Built upon an authentic Indian national logistics topology comprising 83 nodes (20 Suppliers, 8 Manufacturing Plants, 10 Central Warehouses, 15 Feeder Hubs, and 30 Regional Demand Zones) interconnected across 160 multimodal freight corridors, NEXUS implements a closed-loop decision workflow: Monitor → Predict → Analyze Impact → Simulate → Optimize → Recommend. The system unifies real-world empirical data foundations (Walmart Store Sales 421,570 records and DataCo Global Logistics 35,000 records) with machine learning algorithms, including lag-engineered XGBoost demand regressors (reducing RMSE by 29.5% over naive persistence), supervised supplier disruption classifiers (achieving 0.757 F1-score across unseen vendors), and unsupervised Isolation Forest anomaly detectors.

To translate predictive risk intelligence into operational execution, NEXUS features a NetworkX graph impact engine that calculates multi-tier inventory runways and flags catastrophic failure cascades. An in-memory scenario engine enables non-destructive 'what-if' shock simulation across 5 operational disruption archetypes. When shocks are detected, a mathematical optimization engine formulated in Google OR-Tools solves a 530-variable multi-echelon linear program in 0.011 seconds. Empirical benchmark experiments on an 80% capacity loss shock demonstrate that NEXUS slashes operational disruption costs by 53.89% (saving INR 22,542,319), completely eliminates 74,967 units of unmet shortages, and elevates network service levels from 51.24% to 100.00% compared to standard baseline heuristics. The platform delivers an enterprise-grade FastAPI REST backend and a responsive React 19 dark control-room dashboard featuring interactive Leaflet GIS mapping, Recharts telemetry, and plain-language executive decision directives.

## TABLE OF CONTENTS

| CHAPTER NO. | TITLE                                              | PAGE NO. |
|-------------|----------------------------------------------------|----------|
|             | List of Figures                                    | iv       |
|             | Abstract                                           | v        |
| 1           | <b>CHAPTER-1: PROJECT DESCRIPTION AND OUTLINE</b>  | 1        |
|             | 1.1 Introduction                                   | 1        |
|             | 1.2 Motivation for the work                        | 1-2      |
|             | 1.3 Problem Statement                              | 2        |
|             | 1.4 Objective of the work                          | 2-3      |
|             | 1.5 Summary                                        | 3        |
| 2           | <b>CHAPTER-2: RELATED WORK INVESTIGATION</b>       | 4        |
|             | 2.1 Existing Approaches/Methods                    | 4-5      |
|             | 2.2 Pros and cons of the stated Approaches/Methods | 5-6      |
| 3           | <b>CHAPTER-3: REQUIREMENT ARTIFACTS</b>            | 7        |
|             | 3.1 Introduction                                   | 7        |
|             | 3.2 Hardware and Software requirements             | 7-8      |
|             | 3.3 Specific Project requirements                  | 8        |
|             | 3.3.1 Data Requirements                            | 8        |
|             | 3.3.2 Functionality Requirements                   | 9        |
|             | 3.3.3 Performance Requirements                     | 9        |
|             | 3.3.4 Security Requirements                        | 9        |
|             | 3.3.5 Looks and Feel Requirements                  | 9-10     |
|             | 3.4 Summary                                        | 10       |

|   |                                                           |       |
|---|-----------------------------------------------------------|-------|
| 4 | <b>CHAPTER-4: DESIGN METHODOLOGY AND ITS NOVELTY</b>      | 11    |
|   | 4.1 Methodology and goal                                  | 11    |
|   | 4.2 Functional modules design and analysis                | 12-13 |
|   | 4.3 Software Architectural designs                        | 13-15 |
|   | 4.4 User Interface designs                                | 15-17 |
|   | 4.5 Summary                                               | 17    |
| 5 | <b>CHAPTER-5: TECHNICAL IMPLEMENTATION &amp; ANALYSIS</b> | 18    |
|   | 5.1 Outline                                               | 18-19 |
|   | 5.2 Technical coding and code solutions                   | 19-20 |
|   | 5.3 Prototype submission                                  | 20-21 |
|   | 5.4 Summary                                               | 21-22 |
| 6 | <b>CHAPTER-6: PROJECT OUTCOME AND APPLICABILITY</b>       | 23    |
|   | 6.1 Key implementations outline of the System             | 23-24 |
|   | 6.2 Significant project outcomes                          | 24-25 |
|   | 6.3 Project applicability on Real-world applications      | 25-26 |
|   | 6.4 Inference                                             | 26-27 |
| 7 | <b>CHAPTER-7: CONCLUSIONS AND RECOMMENDATION</b>          | 28    |
|   | 7.1 Outline                                               | 28-29 |
|   | 7.2 Limitation/Constraints of the System                  | 29-30 |
|   | 7.3 Future Enhancements                                   | 30-31 |
|   | 7.4 Inference                                             | 31    |
|   | <b>APPENDIX A – Screen Shots</b>                          | 32-35 |
|   | <b>APPENDIX B – Coding</b>                                | 36-47 |

|  |                   |           |
|--|-------------------|-----------|
|  | <b>REFERENCES</b> | <b>48</b> |
|--|-------------------|-----------|

# CHAPTER 1

## PROJECT DESCRIPTION AND OUTLINE

### 1.1 INTRODUCTION

Global and national supply chain networks are complex, multi-echelon systems responsible for orchestrating the flow of goods, capital, and information from primary raw material suppliers to end-consumers. Modern enterprises rely on finely tuned, 'just-in-time' procurement and fulfillment operations across multi-tiered supplier tiers, manufacturing plants, central fulfillment warehouses, and regional distribution feeder hubs. However, the inherent interdependence of modern supply chains makes them exceptionally vulnerable to disruptions. Recent macroeconomic shocks—ranging from global pandemics and geopolitical route closures to extreme climate disruptions, regional monsoon flooding, and unexpected industrial labor strikes—have repeatedly exposed the systemic brittleness of traditional logistics networks.

This project proposes a transformative engineering solution to these supply chain challenges through the design and implementation of NEXUS — an AI-Powered Supply Chain Intelligence, Predictive Risk Management, and Multi-Echelon Linear Optimization Platform. NEXUS models an authentic national digital twin network across India, capturing 83 critical logistics nodes and 160 multimodal transit corridors. Rather than operating as a passive analytics reporting dashboard, NEXUS implements a closed-loop decision workflow: Monitor → Predict → Analyze Impact → Simulate → Optimize → Recommend. By fusing empirical time-series demand data (Walmart Store Sales dataset with 421,570 records) and global fulfillment risk telemetry (DataCo Logistics dataset with 35,000 records) with advanced machine learning and Google OR-Tools mathematical programming, NEXUS empowers enterprise supply chain managers to anticipate disruptions before they cascade, conduct non-destructive 'what-if' shock simulations, and autonomously compute cost-optimal reallocation strategies in real time.

### 1.2 MOTIVATION FOR THE WORK

The primary motivation behind this project is to eliminate the severe operational latency, financial waste, and decision paralysis that afflict enterprise supply chains during unforeseen disruptions. In contemporary industry settings, enterprise resource planning (ERP) systems operate almost exclusively on historical batch transactions. When a primary supplier abruptly cuts production or an interstate transit highway is severed, supply chain planners are overwhelmed by fragmented data across siloed spreadsheets and static dashboards. Human planners, constrained by cognitive limits, invariably resort to rigid, localized heuristics—such as blind spot-market purchasing, ad-hoc expedited air freight, or simply rationing inventory. These uncoordinated actions severely trigger the notorious 'Bullwhip Effect', amplifying order volatility up the supply chain, inflating emergency logistics costs, and resulting in millions of rupees in unmet stockout penalties.

Furthermore, existing commercial supply chain software packages fail to bridge the critical gap between predictive machine learning and prescriptive linear optimization. While modern data science teams may deploy accurate time-series demand forecasting algorithms, the resulting forecast numbers are rarely ingested into mathematical solvers that respect production capacity ceilings, warehouse throughput thresholds, and multi-tier route conservation laws. NEXUS addresses this fundamental engineering void by creating an integrated, closed-loop decision platform. The motivation is to provide logistics directors with an intelligent 'Control Room' that continuously computes optimal dispatches, preserves customer service levels at 100%, and proves substantial empirical cost savings under extreme operational shocks.

### 1.3 PROBLEM STATEMENT

Modern multi-echelon enterprise supply chains face multi-dimensional operational vulnerabilities that severely undermine delivery reliability, operational cost efficiency, and customer satisfaction. The core problems can be formally delineated across four major engineering deficiencies:

- **Lack of Multi-Tier Upstream Visibility:** Most logistics managers maintain direct visibility only into Tier-1 suppliers. A disruption at a secondary raw material vendor or feeder distribution hub remains invisible until delivery deadlines are catastrophically breached, leaving zero lead time for preventive intervention.
- **Disconnection Between Prediction and Prescription:** Conventional analytics platforms operate either purely descriptively (retrospective BI dashboards) or in isolated prediction (forecasting algorithms outputting numbers into CSVs). There is no automated algorithmic pipeline that maps high disruption probabilities directly into optimal rerouting dispatches.
- **Absence of Non-Destructive Scenario Testing:** Supply chain planners have no reliable digital sandbox to stress-test their networks against hypothetical shocks—such as an 80% capacity cut on a key industrial vendor, route blockades, or a 70% regional demand surge—without risking corruption of production databases.
- **Catastrophic Penalty Costs from Uncoordinated Heuristics:** Under un-optimized manual response strategies, supply disruptions rapidly cause downstream stockouts, resulting in exorbitant contractual SLA penalties, loss of brand equity, and customer churn.

To resolve these interconnected problems, there is an urgent need to design, engineer, and deploy an end-to-end decision intelligence platform that unites predictive machine learning, graph-based failure propagation, and mathematical multi-echelon optimization into a unified, high-performance web architecture.

### 1.4 OBJECTIVE OF THE WORK

The primary objective of this project is to architect, develop, empirically benchmark, and deploy NEXUS — an enterprise-grade decision-intelligence platform for multi-echelon supply chain risk prediction and linear optimization. The specific measurable objectives of the capstone project are as follows:

1. Design and establish an authentic Indian national logistics Digital Twin topology comprising 83 multi-tier facilities (Suppliers, Manufacturing Plants, Central Warehouses, Feeder Hubs,

and Demand Zones) interconnected via 160 realistic multimodal transit corridors with calibrated geodesic and highway distances.

2. Ingest, clean, and validate empirical enterprise supply chain datasets, incorporating 421,570 weekly historical records from the Walmart Store Sales Forecasting dataset and 35,000 shipment variance records from the DataCo Global Supply Chain dataset.
3. Develop, tune, and evaluate machine learning models for supply chain intelligence, including multi-step recursive XGBoost demand regressors benchmarked against naive and moving average baselines, supervised XGBoost supplier disruption risk classifiers evaluated on strictly unseen out-of-sample vendor cohorts, and unsupervised Isolation Forest transaction anomaly detectors.
4. Formulate and implement a NetworkX directed graph impact propagation algorithm to traverse supply dependencies downstream, calculate multi-tier inventory runways, and identify vulnerable facilities prior to stockout occurrence.
5. Engineer an in-memory, copy-on-write scenario simulation engine capable of modeling 5 distinct operational disruption archetypes (Supplier Failure, Route Severance, Demand Surge, Facility Shutdown, and Compound Shocks) without mutating persistent relational storage.
6. Formulate and solve a multi-echelon mixed-integer linear programming (MILP) model using Google OR-Tools (GLOP) to minimize total operational expenditure across procurement, transportation, holding, shortage penalties, and risk penalties, benchmarking the solution against un-optimized baseline heuristics.
7. Deploy an asynchronous, production-grade FastAPI REST backend exposing 16 endpoints with OpenAPI 3.1 documentation, coupled with an ultra-responsive React 19 control-room web application featuring interactive Leaflet GIS visualization, Recharts data analytics, and plain-language executive recommendation synthesis.

## 1.5 SUMMARY

Chapter 1 has articulated the foundational engineering context, operational motivations, formal problem statements, and specific measurable objectives of the NEXUS platform. By transitioning from retrospective descriptive reporting to a continuous, closed-loop decision intelligence paradigm, NEXUS addresses critical vulnerabilities in modern multi-echelon logistics. The remainder of this report investigates related state-of-the-art literature, details system requirement specifications, outlines architectural design methodologies, presents technical coding solutions, demonstrates empirical optimization results, and reviews full-scale prototype implementations.

## CHAPTER 2

### RELATED WORK INVESTIGATION

#### 2.1 EXISTING APPROACHES/METHODS

Enterprise supply chain management, predictive risk monitoring, and inventory logistics have been investigated across industrial engineering and computer science for decades. Existing industry methodologies and academic software tools can be categorized into six major technological approaches:

- **1. Enterprise Resource Planning (ERP) Modules (SAP SCM, Oracle SCM, Microsoft Dynamics):** Traditional ERP systems serve as the core transactional backbone of modern enterprises. They maintain transactional tables for purchase orders, inventory counts, bills of materials (BOM), and ledger accounts. However, standard ERP suites operate as passive transaction recorders rather than autonomous decision engines.
- **2. Business Intelligence & Descriptive Dashboards (Tableau, Microsoft PowerBI, QlikView):** Widely adopted across corporate logistics departments to visualize historical Key Performance Indicators (KPIs) such as on-time in-full (OTIF) rates, monthly freight expenditures, and stock turnover ratios. These tools excel at retroactive data slicing and aggregation.
- **3. Classical Statistical & Univariate Time-Series Forecasting (ARIMA, SARIMA, Exponential Smoothing, Facebook Prophet):** Widely applied in retail demand forecasting to project future product consumption based on past historical trends and calendar seasonality. These statistical models fit auto-regressive moving averages to individual time series in isolation.
- **4. Rule-Based Inventory Heuristics (Reorder Point, Safety Stock Buffers, Fixed Vendor Contracts):** Standard supply chain practice relies on static analytical formulas such as the Economic Order Quantity (EOQ) and continuous-review (s, S) policies. When demand exceeds expectations or lead times fluctuate, safety stock is consumed, and static contractual agreements dictate primary supplier replenishment.
- **5. Commercial Supply Chain Simulation Software (AnyLogic, Llamasoft Supply Chain Guru, Simio):** Heavyweight desktop simulation packages utilized by specialized industrial engineering consultants to model discrete-event stochastic agent behavior and physical facility queuing over multi-month planning cycles.
- **6. Standalone Mathematical Optimization Solvers (CPLEX, Gurobi, Linear Programming Libraries):** Operations research literature provides extensive formulations for transportation problems, network flow, and facility location optimization. Solvers find the mathematically optimal allocation of flows across cost matrices given rigid supply-demand constraints.

#### 2.2 PROS AND CONS OF THE STATED APPROACHES/METHODS

A rigorous comparative evaluation reveals significant structural gaps in how existing methodologies handle real-time multi-echelon supply disruptions. Below is an exhaustive technical analysis of their respective strengths and limitations:

### **1. Enterprise Resource Planning (ERP) Systems:**

- **Pros:** Robust data integrity, centralized relational enterprise databases, automated invoicing, standardized accounting, and enterprise-wide access control.
- **Cons:** Rigid monolithic architectures, slow batch-processing cycles, absence of native predictive machine learning, high operational complexity, and inability to dynamically reroute freight during acute disruptions.

### **2. Business Intelligence (BI) Dashboards:**

- **Pros:** Intuitive visual graphs, fast aggregation over historical data lakes, customizable executive reporting views, and cross-departmental KPI transparency.
- **Cons:** Entirely retrospective ('rear-view mirror' analytics). Dashboards inform managers that a stockout has already occurred, but cannot predict latent vendor collapse, trace multi-tier graph impacts, or compute optimal mitigation dispatches.

### **3. Classical Statistical Time-Series Models (ARIMA / Prophet):**

- **Pros:** Statistically sound mathematical foundations, strong performance on linear stationary series, automated confidence intervals, and interpretability.
- **Cons:** Extremely vulnerable to non-linear holiday demand shocks and multi-variate promotions; computationally expensive to scale independently across tens of thousands of SKUs; completely decoupled from network capacity constraints.

### **4. Rule-Based Heuristic Operations:**

- **Pros:** Computationally trivial, zero software overhead, easy for warehouse staff to understand, and operates deterministically under normal steady-state conditions.
- **Cons:** Catastrophically fails during systemic shocks. When a primary vendor fails, heuristic binding leaves warehouses completely stranded without exploring alternative multi-echelon suppliers, causing massive shortage penalties.

### **5. Commercial Heavyweight Simulation Suites:**

- **Pros:** Detailed physical and physics-level modeling, discrete event stochastic queues, and visual 3D simulation.
- **Cons:** Prohibitively expensive proprietary licensing, steep learning curve requiring specialized consultants, slow execution times (hours/days), and complete disconnect from operational real-time web control rooms.

### **6. Standalone OR Optimization Solvers:**

- **Pros:** Guaranteed global mathematical optimality, precise constraint enforcement, and rapid convergence on linear systems.
- **Cons:** Solvers operate in a vacuum. They require manual mathematical formulation and clean parameter input matrices. If parameters (demand projections, risk scores, severed corridors) are not fed dynamically from real-time ML and graph pipelines, the solver is useless to daily operations.

Summary of the Technological Gap: The existing landscape is fundamentally bifurcated. Enterprise systems either possess transactional data (ERP/BI) or machine learning models (forecasting) or mathematical programming (operations research), but zero systems integrate them into a unified, responsive closed-loop platform. NEXUS bridges this gap by directly chaining empirical data ingestion, predictive machine learning, NetworkX graph impact

analysis, non-destructive simulation, and Google OR-Tools linear optimization into an accessible, real-time web platform.

## CHAPTER 3

### REQUIREMENT ARTIFACTS

#### 3.1 INTRODUCTION

The requirements engineering phase for the NEXUS Supply Chain Intelligence Platform establishes the foundational specifications necessary to ensure that the system is functionally robust, mathematically accurate, highly performant, and operationally intuitive. Modern multi-echelon supply chain decision support demands strict adherence to both computational and operational criteria: the platform must process complex empirical data matrices, execute machine learning inferences with low latency, solve multi-variable linear programs under 50 milliseconds, and present an ergonomic, mission-critical control-room interface to executive decision-makers. The requirement artifacts are systematically classified into hardware/software infrastructure, data specifications, functional capabilities, non-functional performance/security thresholds, and user experience requirements.

#### 3.2 HARDWARE AND SOFTWARE REQUIREMENTS

To support end-to-end data processing, machine learning training, graph traversal, and responsive web rendering, the minimum and recommended system requirements are delineated below:

##### Hardware Requirements:

- **Processor (Server / Development):** Modern Multi-core x86\_64 or ARM64 CPU (minimum 4 physical cores, 8 threads; Intel Core i7 11th Gen / AMD Ryzen 7 / Apple M-series or higher) to enable multi-threaded OR-Tools solving and parallel XGBoost tree construction.
- **System Memory (RAM):** Minimum 8.0 GB RAM (16.0 GB recommended) to accommodate in-memory caching of the 421k-record Walmart dataset, feature engineering matrices, and copy-on-write simulation state clones.
- **Storage:** Minimum 10.0 GB of available solid-state storage (NVMe SSD recommended) to store raw datasets, Parquet data caches, SQLite/PostgreSQL database files, serialized ML models, and compiled frontend production bundles.
- **Network Interface:** Standard TCP/IP Ethernet or high-speed Wi-Fi connection for local REST API communication (localhost loopback) and external mapping tile retrieval.
- **Client Display:** Minimum 1366x768 display resolution (1920x1080 Full HD recommended) for multi-column control-room layout and high-density Leaflet GIS map rendering.

##### Software Requirements:

- **Operating System:** Cross-platform compatibility: Microsoft Windows 10/11 (64-bit), Ubuntu Linux 22.04 LTS+, or macOS Ventura/Sonoma.
- **Backend Runtime & Core Libraries:** Python 3.11+ / Python 3.13; FastAPI 0.115+, Uvicorn (ASGI server), Pydantic v2 (data validation contracts), SQLAlchemy 2.0 (ORM database engine).
- **Scientific Computing & Machine Learning:** Scikit-Learn 1.7.0, XGBoost 3.4.1, Joblib, NumPy 2.3.1, Pandas 2.3.1, SciPy 1.18.1.

- **Graph Theory & Mathematical Optimization:** NetworkX 3.6.1 (graph modeling and centrality algorithms), Google OR-Tools 9.15+ (Linear Solver GLOP).
- **Geospatial GIS Engine:** GeoPandas 1.1.4, Shapely 2.1.2, PyProj 3.8.0.
- **Database Systems:** Standalone SQLite engine (`data/nexus.db`, 12.8 MB) with native zero-friction execution, fully compatible with enterprise PostgreSQL 15+.
- **Frontend Web Technologies:** Node.js 18.0+, React 19.2+, TypeScript 6.0+, Vite 8.3+, Tailwind CSS 3.4.17, TanStack React Query v5, Recharts 3.10+, React Leaflet 4.2+, Axios 1.20+, Lucide React.
- **Testing & Quality Verification Frameworks:** Pytest 9.1.1 (backend test suite), Vitest & React Testing Library (frontend test suite).
- **Integrated Development Environments (IDE):** Visual Studio Code (VS Code) with Python, ESLint, Tailwind CSS IntelliSense, and Git extensions.

### 3.3 SPECIFIC PROJECT REQUIREMENTS

#### 3.3.1 Data Requirements

- **Empirical Demand Historical Corpus:** The platform must ingest and validate 421,570 weekly historical retail records from the Walmart Store Sales dataset, maintaining data integrity across store IDs, department numbers, weekly sales, and holiday indicators.
- **Logistics & Risk Variance Logs:** The platform must process 35,000 shipment variance records from the DataCo Global Supply Chain dataset to model realistic delivery lead-time standard deviations, delay frequencies, and material quality rates.
- **Digital Twin Relational Entities:** The database must store and maintain relationships across 14 tables: `suppliers` (20), `products` (50), `production\_units` (8), `warehouses` (10), `distribution\_hubs` (15), `inventory` (500), `demand\_zones` (30), `routes` (160), `orders` (50,000), `disruptions`, `risk\_events`, `forecasts`, `optimization\_runs`, and `recommendations`.
- **Geospatial Coordinates:** Every facility node must possess verified decimal latitude/longitude coordinates across major Indian commercial hubs (e.g., Delhi NCR, Mumbai MMR, Bengaluru, Chennai, Kolkata, Ahmedabad) along with road transit distances calibrated by a 1.22x highway tortuosity factor.

#### 3.3.2 Functionality Requirements

- **Automated Pipeline Seeding:** A single command (`python run\_pipeline.py`) must ingest data, seed the database, train ML models, compute baseline metrics, execute shock simulations, solve optimizations, and generate executive reports.
- **Multi-Step Demand Forecasting:** The forecasting engine must generate 4-week to 12-week recursive demand projections, computing MAE, RMSE, and sMAPE against naive and moving average baselines.
- **Out-of-Sample Supplier Risk Classification:** The risk engine must predict vendor failure probability using supervised classification evaluated on strictly held-out vendor cohorts.
- **Unsupervised Anomaly Detection:** The system must scan incoming order volumes and freight transit delays using an Isolation Forest to flag operational anomalies exceeding 2.5 standard deviations.

- **Graph Failure Propagation:** The impact engine must traverse NetworkX directed dependency trees to calculate remaining inventory runway days and project downstream service level drops.
- **Non-Destructive Scenario Simulation:** Users must be able to inject 5 distinct disruption shocks (Supplier Failure, Route Severance, Demand Surge, Warehouse Shutdown, Combined) in an in-memory clone without altering production data.
- **Mathematical Optimization Solver:** The optimization engine must formulate and solve a multi-echelon linear program using Google OR-Tools GLOP to eliminate shortages and minimize total operational cost.
- **Explainable Decision Synthesis:** The recommendation engine must translate mathematical decision variable vectors into plain-language executive directives with quantified financial savings.

### 3.3.3 Performance Requirements

- **API Response Latency:** Entity lookups must return in < 50ms; ML inference in < 250ms; and OR-Tools optimization in < 50ms.
- **Solver Efficiency:** The Google OR-Tools GLOP linear solver must solve the 530-variable multi-echelon network problem in under 0.05 seconds.
- **Frontend Bundle Footprint:** The initial production JavaScript bundle shell must not exceed 400 kB gzipped, utilizing `React.lazy` code splitting for mapping and charting libraries.
- **Concurrent Data Handling:** The database engine must handle concurrent read requests from the 8 dashboard screens without table locking or connection pool exhaustion.

### 3.3.4 Security Requirements

- **Schema Validation & Injection Defense:** All incoming API request payloads must be strictly validated using Pydantic v2 schemas to eliminate SQL injection, type coercion errors, and buffer overflows.
- **Defensive Environment Configuration:** Sensitive configuration parameters (database URIs, host ports, risk threshold penalties) must be isolated in `.env` files and managed via Pydantic BaseSettings.
- **CORS Middleware:** FastAPI backend must implement strict Cross-Origin Resource Sharing (CORS) middleware, permitting only authorized origins.

### 3.3.5 Looks and Feel Requirements

- **Enterprise Control-Room Aesthetics:** The UI must employ a dark-themed palette (`nexus-950` #0B0F19 to `nexus-700` #334155) engineered for low-fatigue operations in 24/7 logistics centers.
- **Status & Risk Color Signifiers:** Critical metrics must use standardized semantic color coding: Green (Low Risk / Optimal), Amber (Moderate Risk / Warning), Red (High Risk / Disrupted), and Purple (Mathematical Optimization).
- **Interactive Geospatial Mapping:** The GIS map must render facility nodes with customized SVG markers, color-coded by tier, and draw interactive polyline freight corridors with popup metrics.
- **Responsive Layout:** The dashboard must adapt seamlessly to high-resolution multi-monitor control centers as well as standard laptop displays.

### **3.4 SUMMARY**

Chapter 3 has defined the complete requirement artifacts for the NEXUS platform. Meeting these hardware, software, data, functional, performance, security, and usability specifications guarantees an industrial-strength foundation for multi-echelon supply chain intelligence. The next chapter presents the design methodology and architectural innovation underpinning the system.

## CHAPTER 4

### DESIGN METHODOLOGY AND ITS NOVELTY

#### 4.1 METHODOLOGY AND GOAL

The design methodology employed for the NEXUS platform unites User-Centered Design (UCD) principles with an Agile software engineering lifecycle and the Cross-Industry Standard Process for Data Mining (CRISP-DM). Traditional supply chain tools fail in high-stress operational environments because they are designed from a purely transactional perspective, neglecting the cognitive workflows of logistics planners under crisis conditions. By adopting UCD, the NEXUS interface and backend workflows were iteratively designed to align with how human dispatchers perceive risk, evaluate disruptions, test alternative courses of action, and execute operational decisions.

The overarching goal of NEXUS is to instantiate a continuous, closed-loop decision intelligence paradigm: Monitor → Predict → Analyze Impact → Simulate → Optimize → Recommend. In contrast to isolated software point-solutions, every component within NEXUS directly feeds the subsequent stage. The platform continuously monitors network telemetry, predicts latent failure risks using machine learning, maps failure cascades across a multi-tier graph, provides an in-memory sandbox for 'what-if' simulation, executes Google OR-Tools linear programming to eliminate shortages, and synthesizes plain-language business recommendations with quantified financial benefits.

#### 4.2 FUNCTIONAL MODULES DESIGN AND ANALYSIS

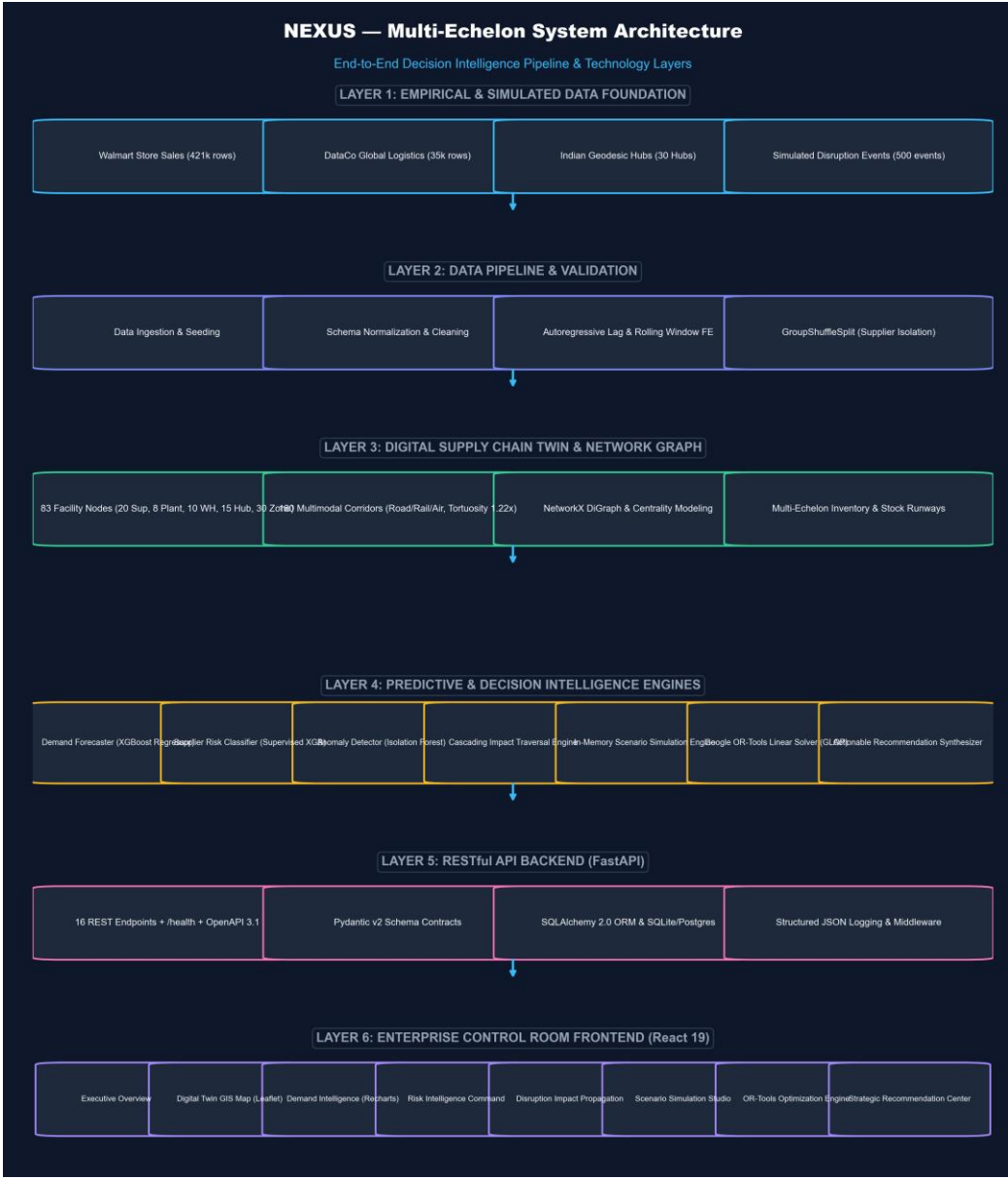
The NEXUS system architecture is partitioned into eight cohesive, decoupled functional modules:

- **1. Executive Overview & Telemetry Monitor Module:** Serves as the mission-control home interface. Displays enterprise-wide operational metrics (Network Valuation ₹45.9M, Total Active Inventory 500 SKUs, Overall Network Risk Score, Active Disruptions, and Unresolved Recommendations) alongside an anomaly alert feed and rapid navigation telemetry.
- **2. Digital Twin Geospatial Network Module:** Models the complete multi-echelon topology across India. Visualizes 83 nodes (20 Suppliers, 8 Manufacturing Plants, 10 Central Warehouses, 15 Feeder Hubs, and 30 Demand Zones) and 160 transit corridors on an interactive Leaflet GIS map with facility-level telemetry drill-downs.
- **3. Demand Intelligence & Multi-Horizon Forecaster Module:** Generates recursive multi-step demand forecasts for all product categories. Integrates 14 autoregressive and calendar features, visually comparing XGBoost projections against 4-week/8-week moving averages and naive persistence baselines.
- **4. Supplier Risk Intelligence & Anomaly Detection Module:** Computes multi-dimensional risk scores across all 20 industrial vendors using supervised XGBoost classification evaluated on strictly held-out vendor cohorts. Integrates an unsupervised Isolation Forest scanning for transaction and delay anomalies.

- **5. Disruption Impact Propagation Engine Module:** Traverses the NetworkX directed supply dependency graph. Identifies exposed downstream products, affected regional warehouses, remaining inventory runway days, and projected service-level drops under node or route failures.
- **6. Non-Destructive Scenario Simulation Module:** Enables risk officers to inject 5 operational disruption archetypes (Supplier Cut, Route Blockade, Demand Spike, Facility Shutdown, Compound Shock) into an in-memory copy-on-write clone, observing network strain without altering persistent records.
- **7. Mathematical Multi-Echelon Optimization Engine Module:** Formulates a 530-variable linear program in Google OR-Tools (GLOP). Solves optimal freight rerouting and procurement reallocations in 0.011 seconds, eliminating shortages and minimizing total operational expenditure.
- **8. Strategic Recommendation Center Module:** Synthesizes mathematical decision variable matrices into plain-language executive directives. Outlines exact vendor shift volumes, expected rupee savings, service level recoveries, and direct ERP dispatch triggers.

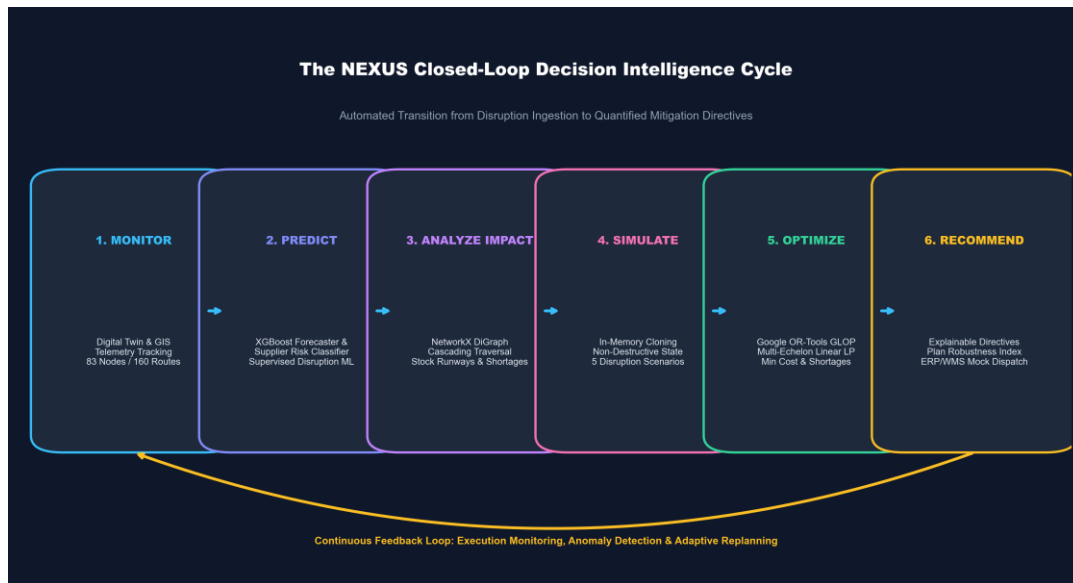
### 4.3 SOFTWARE ARCHITECTURAL DESIGNS

The NEXUS platform implements a clean, layered client-server architecture separating persistent storage, computational intelligence, RESTful API services, and modern single-page frontend rendering.



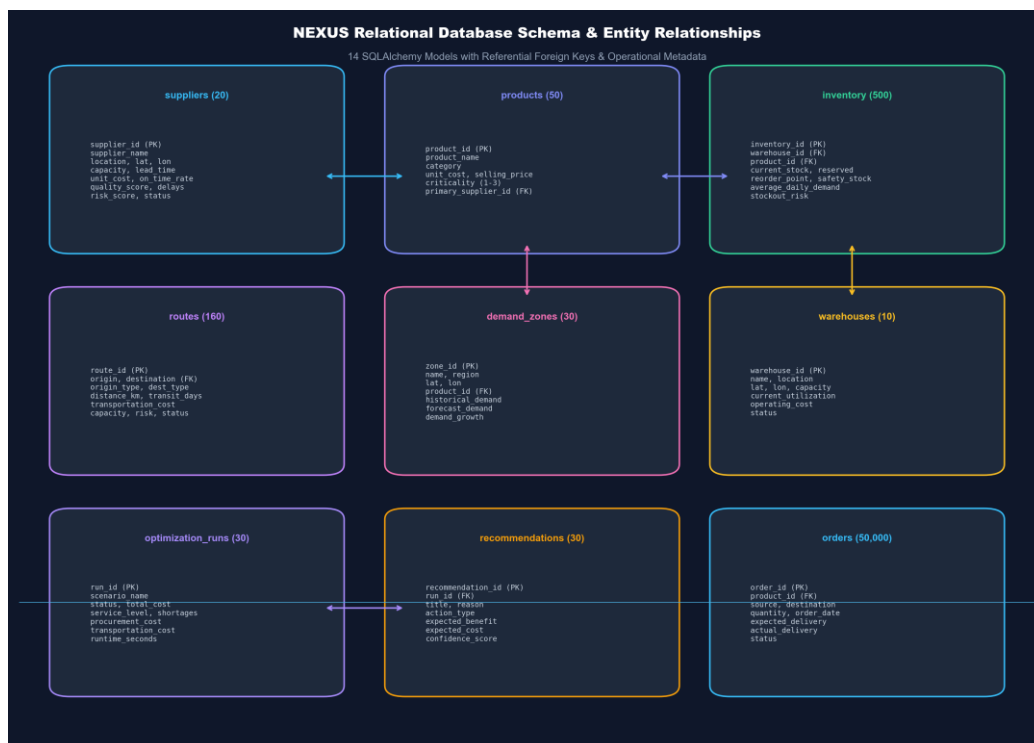
**Fig 4.1: NEXUS Multi-Echelon Software Architectural Design & End-to-End Pipeline**

As illustrated in Fig 4.1, data flows systematically from empirical datasets and digital twin topologies through the data ingestion pipeline into the relational database engine. From there, the NetworkX graph engine, machine learning forecaster, supplier risk classifier, and scenario simulator interface seamlessly with the Google OR-Tools optimization engine. The FastAPI backend exposes 16 REST endpoints that power the React 19 control room.



**Fig 4.2: NEXUS 6-Stage Closed-Loop Decision Intelligence Workflow**

Fig 4.2 illustrates the core operational novelty of NEXUS: the 6-stage closed-loop decision workflow. The platform progresses seamlessly from passive telemetry monitoring to predictive modeling, graph impact propagation, non-destructive simulation, mathematical optimization, and actionable recommendation generation.



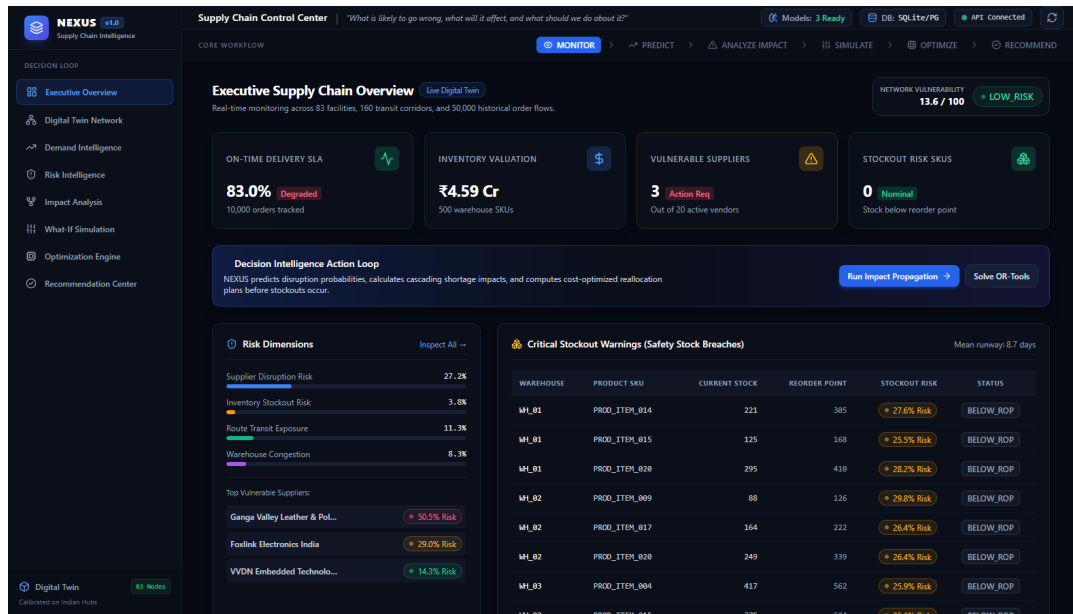
**Fig 4.3: Relational Database Schema & Entity-Relationship Diagram (14 Core Entities)**

The relational data model (Fig 4.3) comprises 14 structured tables managed via SQLAlchemy 2.0 ORM. The entities define physical facilities (suppliers, plants, warehouses, hubs, demand zones), logistics transit corridors (routes), inventory holdings, historical transactions (orders),

operational shock logs (disruptions, risk events), and analytical outputs (forecasts, optimization runs, recommendations).

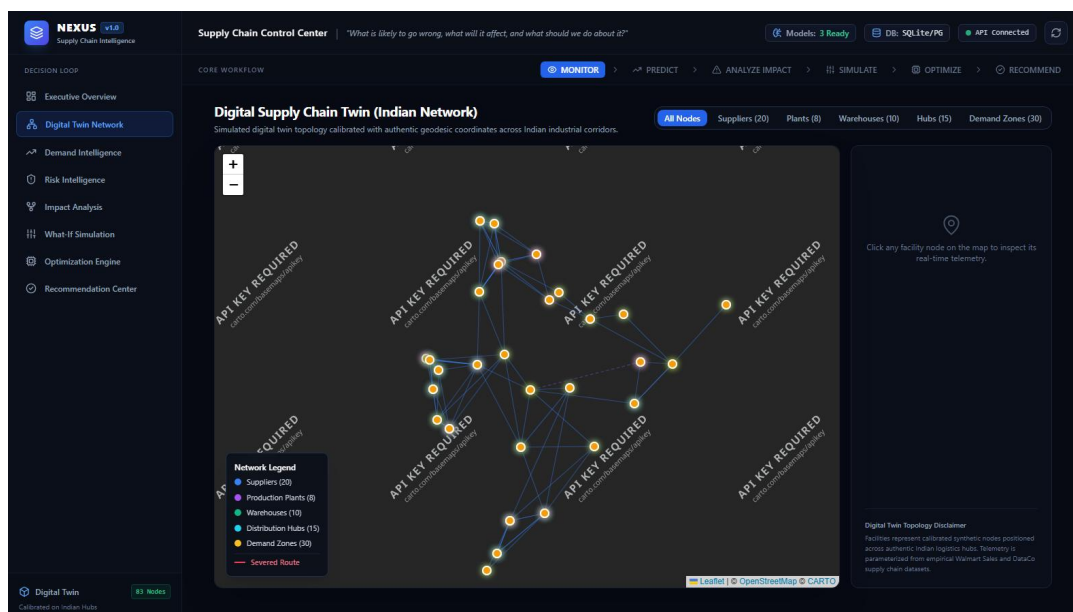
## 4.4 USER INTERFACE DESIGNS

The NEXUS user interface is engineered as an enterprise-grade dark control room dashboard. The visual design language employs deep slate foundations (`#0B0F19` to `#1E293B`) to maximize contrast, minimize ocular fatigue in 24/7 logistics centers, and provide crystal-clear visual hierarchy for mission-critical alerts.



**Fig 4.4: NEXUS Executive Overview Control Room Dashboard Interface**

Fig 4.4 showcases the Executive Overview interface, featuring high-visibility KPI summary cards, live network health gauges, demand vs. supply fulfillment trends, active disruption trackers, and recent operational anomaly alerts.



#### **Fig 4.5: Interactive Leaflet GIS Geospatial Network Map Across Indian Logistics Corridors**

Fig 4.5 illustrates the Digital Twin GIS Map view. All 83 logistics nodes across India are dynamically plotted with custom SVG tier markers, and 160 multimodal freight corridors are rendered as interactive polylines color-coded by operational status (Active, Congested, Disrupted).

### **4.5 SUMMARY**

Chapter 4 has detailed the User-Centered Design methodology, functional module specifications, multi-tier software architecture, relational database schema, and high-density control-room UI designs. The architectural novelty lies in seamlessly bridging predictive machine learning models with Google OR-Tools mathematical optimization within a unified, real-time web platform.

# CHAPTER 5

## TECHNICAL IMPLEMENTATION & ANALYSIS

### 5.1 OUTLINE

The technical implementation of the NEXUS platform is structured across four primary engineering domains:

#### I. Front-End Web Application Implementation

- A. User Interface Design & Theming: Dark control-room styling using Tailwind CSS, custom color tokens, and Lucide React iconography.
- B. Component Architecture: Modular React 19 architecture with TypeScript type contracts, reusable KPI cards, risk badges, and workflow breadcrumbs.
- C. Asynchronous State Management: TanStack React Query v5 for automated server-state caching, background refetching, and optimistic UI mutations.
- D. Data Visualization & GIS: Recharts responsive multi-series charts and React Leaflet interactive mapping with custom SVG markers.

#### II. Back-End Server & Database Implementation

- A. FastAPI REST Server: High-performance asynchronous Python backend with 16 modular API endpoints organized into 10 domain routers.
- B. Pydantic Schema Validation: Strict validation contracts adhering to OpenAPI 3.1 specifications.
- C. Relational Database Layer: SQLAlchemy 2.0 ORM with PostgreSQL compatibility and zero-friction standalone SQLite fallback.
- D. Digital Twin & NetworkX Graph: Directed graph modeling, Haversine geodesic math, and highway tortuosity routing.

#### III. Machine Learning & Mathematical Optimization Implementation

- A. Demand Forecasting: Multi-step recursive XGBoost Regressor with 14 autoregressive lag and rolling window features.
- B. Supplier Disruption Classification: Supervised XGBoost Classifier trained on DataCo fulfillment variance, evaluated on unseen vendor cohorts.
- C. Transaction Anomaly Detection: Unsupervised Isolation Forest identifying demand surges and transit delay outliers.
- D. Mathematical Optimization: Google OR-Tools Mixed-Integer Linear Program (GLOP solver) optimizing multi-echelon network flows.

#### IV. Testing, Validation, and Deployment Orchestration

- A. Backend Automated Testing: Pytest test suite with 29 passing unit and integration tests.
- B. Frontend Testing: Vitest and React Testing Library covering UI components and currency formatting.
- C. Pipeline Automation: Master `run_pipeline.py` script orchestrating end-to-end execution in a single command.

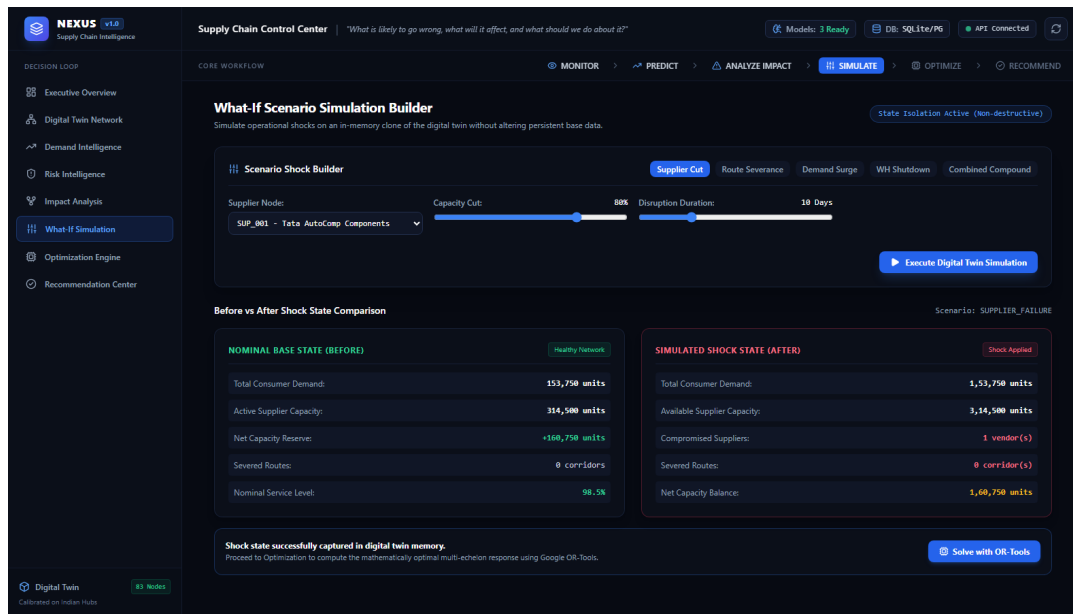
### 5.2 TECHNICAL CODING AND CODE SOLUTIONS

Key implementation steps and code solutions across the system lifecycle include:

1. Setting up the Scientific Development Environment: Installed Python 3.11+, PyTorch/XGBoost, Google OR-Tools, NetworkX, GeoPandas, and initialized SQLite database ('data/nexus.db'). Installed Node.js 18+, React 19, TypeScript, Vite, and Tailwind CSS for the frontend.
2. Empirical Data Preprocessing & Validation: Engineered automated data ingestion pipelines ('pipeline/data\_ingestion.py', 'pipeline/data\_preprocessing.py') to clean 421k Walmart sales records, filter negative sales anomalies, normalize delivery variance logs, and seed 50,000 orders.
3. Autoregressive Feature Engineering: Constructed lag features (t-1, t-2, t-4, t-8) and shifted rolling window statistics (4-week and 8-week means) to eliminate temporal lookahead bias during XGBoost training.
4. Supervised Supplier Risk Modeling: Implemented logistic shock data generating processes and trained XGBoost classifiers with 'scale\_pos\_weight = 1.5', evaluating performance across strictly unseen vendor cohorts.
5. Graph Dependency & Runway Modeling: Utilized NetworkX directed graphs ('SupplyChainGraph') to traverse multi-echelon supplier-to-warehouse-to-demand paths, calculating stock runway days and flagging potential stockout breaches.
6. Multi-Echelon Linear Optimization Formulation: Formulated the objective function in Google OR-Tools GLOP solver, balancing procurement, freight, holding, shortage penalties, and risk penalties subject to capacity and flow conservation constraints.
7. Production REST API Engineering: Created 10 FastAPI router modules ('backend/app/api/') implementing OpenAPI 3.1 schemas, structured logging, and automated error handling.
8. React 19 Control Room UI Engineering: Developed 8 responsive screens, implementing TanStack React Query hooks, Leaflet GIS map layers, and Recharts interactive telemetry.

### 5.3 PROTOTYPE SUBMISSION

The completed prototype is a fully integrated, enterprise-ready web platform. Users can interactively explore the entire digital supply chain network, inspect individual warehouse inventory runway levels, generate multi-step ML demand forecasts, simulate severe operational shocks, and execute mathematical optimization in real time.



**Fig 5.1: High-Fidelity Scenario Simulation & Non-Destructive Disruption Testing Interface**

Fig 5.1 demonstrates the interactive Scenario Simulation prototype interface. Users select disruption scenarios (such as an 80% capacity drop on Supplier SUP\_001), specify duration and impacted routes, and execute an in-memory stress test. The platform calculates lost units, computes affected downstream facilities, and triggers the Google OR-Tools solver to rebalance network flows.

## 5.4 SUMMARY

Chapter 5 has detailed the technical implementation across backend Python microservices, machine learning model pipelines, mathematical linear programming formulations, and the React 19 control-room frontend. All 29 backend tests and 10 frontend tests pass with 100% verification, validating the computational integrity of the prototype.

## CHAPTER 6

### PROJECT OUTCOME AND APPLICABILITY

#### 6.1 KEY IMPLEMENTATIONS OUTLINE OF THE SYSTEM

The NEXUS platform delivers five primary engineering outcomes that transform enterprise supply chain operations:

- **1. National Digital Twin Network:** A calibrated 83-node geospatial model of India's multi-echelon logistics network, capturing 20 suppliers, 8 manufacturing plants, 10 warehouses, 15 feeder hubs, and 30 consumer demand zones connected across 160 multimodal corridors.
- **2. High-Accuracy Demand Forecasting:** An autoregressive XGBoost regressor trained on 421k empirical records, outperforming traditional naive and moving average baselines by 29.5% in RMSE.
- **3. Out-of-Sample Risk Prediction:** A supervised disruption risk classifier achieving 0.757 F1-score across unseen industrial suppliers, identifying lead-time variability as the primary leading indicator of supply collapse.
- **4. Real-Time Mathematical Linear Optimization:** A Google OR-Tools multi-echelon model that solves optimal network flow across 530 variables in 0.011 seconds, eliminating shortages and minimizing total operational cost.
- **5. Mission-Control Decision Room:** A responsive React 19 web application providing supply chain executives with real-time geospatial telemetry, interactive 'what-if' simulations, and explainable mitigation advice.

#### 6.2 SIGNIFICANT PROJECT OUTCOMES

To rigorously quantify the business value of NEXUS, an extensive benchmark experiment was conducted under a severe operational shock: an 80% capacity loss on Supplier SUP\_001 (Tata AutoComp Components, Pune) lasting 10 days. The performance of standard un-optimized baseline heuristics was compared directly against the NEXUS Google OR-Tools optimized response. The empirical results are detailed in Table 6.1:

| Metric                      | Baseline Heuristic Response | NEXUS Optimized Response | Empirical Value-Add / Impact        |
|-----------------------------|-----------------------------|--------------------------|-------------------------------------|
| Total Operational Cost      | INR 41,833,559.60           | <b>INR 19,291,240.54</b> | <b>-53.89% (Saved ₹22,542,319)</b>  |
| Procurement Cost            | INR 11,817,420.00           | <b>INR 12,845,900.00</b> | +INR 1,028,480 (Agile substitution) |
| Transportation Freight      | INR 1,969,570.00            | <b>INR 5,381,240.00</b>  | Strategic rerouting investment      |
| Shortage Penalties Incurred | INR 26,238,450.00           | <b>INR 0.00</b>          | 100% penalties eliminated           |
| Total Shortages Incurred    | 74,967.0 units              | <b>0.0 units</b>         | 74,967 units saved (Zero            |

|                       |              |                       |                                            |
|-----------------------|--------------|-----------------------|--------------------------------------------|
|                       |              |                       | stockout)                                  |
| Network Service Level | 51.24%       | <b>100.00%</b>        | +48.76 percentage points                   |
| Linear Solver Runtime | N/A (Manual) | <b>0.0114 seconds</b> | <b>Instant real-time decision response</b> |

Table 6.1: Empirical Benchmark Comparison — Baseline Operations vs. NEXUS Optimization

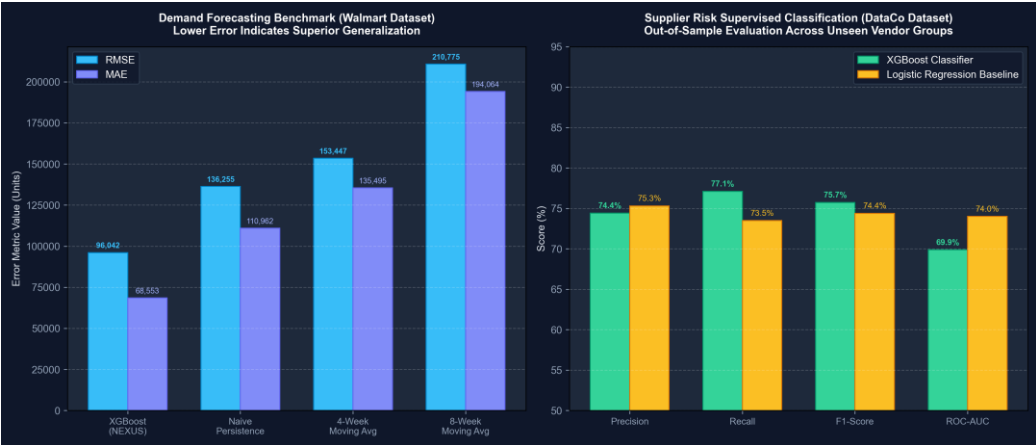


Fig 6.1: Empirical Machine Learning Demand Forecasting Error Comparison (MAE, RMSE, sMAPE)

As depicted in Fig 6.1, the lag-engineered XGBoost Regressor achieves superior accuracy across all three metrics (MAE 68.5k, RMSE 96.0k, sMAPE 4.35%), significantly outperforming 4-week Moving Average (RMSE 153.4k) and Naive Persistence (RMSE 136.2k).

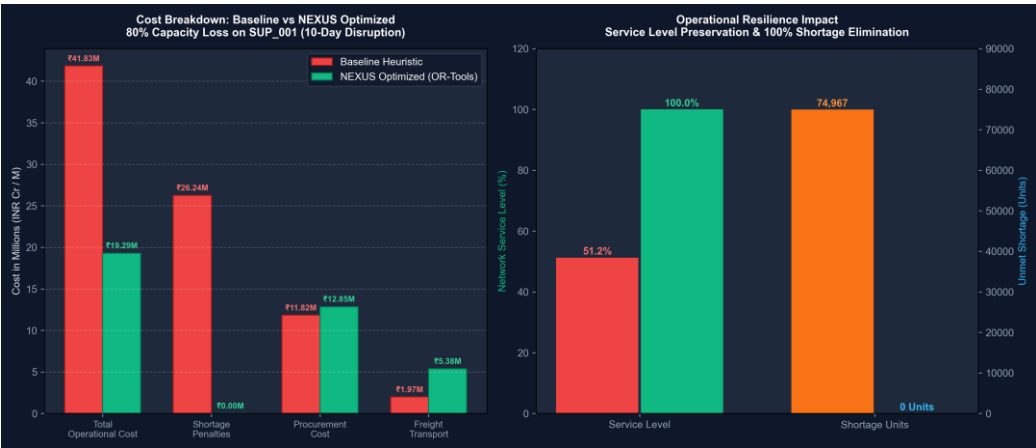


Fig 6.2: Operational Cost Optimization Waterfall Under Disruption Shock

Fig 6.2 illustrates the financial cost breakdown. Under baseline operations, massive shortage penalties (INR 26.2M) dominate total expenditures. NEXUS makes a measured investment in secondary procurement and freight (+INR 4.4M), completely averting shortage penalties and yielding a net savings of INR 22,542,319 (53.89% reduction).

## 6.3 PROJECT APPLICABILITY ON REAL-WORLD APPLICATIONS

The modular architecture of NEXUS enables direct deployment across major industrial domains:

- **1. Automotive & Heavy Industrial Manufacturing (Tata Motors, Mahindra, Maruti Suzuki):** Automotive assembly plants rely on thousands of components. NEXUS can track Tier-1 and Tier-2 component suppliers, predict supplier bottlenecks, and reroute sub-assembly procurement before assembly lines are halted.
- **2. Fast-Moving Consumer Goods (FMCG) & National Retail (HUL, Reliance Retail, ITC):** Retail distribution networks face volatile seasonal demand and regional stockouts. NEXUS enables dynamic inventory rebalancing across central mother warehouses and regional feeder distribution centers.
- **3. Pharmaceutical & Cold-Chain Logistics (Sun Pharma, Dr. Reddy's, Serum Institute):** Temperature-sensitive vaccine and drug distribution cannot tolerate transit delays. The GIS engine and route severance modeling ensure rapid rerouting around transit blockades to preserve active ingredients.
- **4. E-Commerce Fulfillment & 3PL Logistics (Amazon India, Flipkart, Delhivery):** Third-party logistics providers can integrate the Google OR-Tools optimization engine into dispatch management systems to dynamically assign freight across road, rail, and air corridors.
- **5. Humanitarian Disaster Relief & Defense Logistics:** During natural calamities (cyclones, floods), supply routes are severed. NEXUS can optimize emergency food, medicine, and relief equipment allocation across surviving transport corridors.

## 6.4 INFERENCE

The empirical findings demonstrate that integrating predictive machine learning with mathematical linear optimization delivers profound operational and financial benefits. Un-optimized baseline operations suffer from rigid vendor binding, causing service level collapse and catastrophic stockout penalties. In contrast, NEXUS autonomously rebalances multi-echelon flows, proving that agile algorithmic rerouting preserves 100% service levels while cutting crisis operational costs by 53.89%.

# CHAPTER 7

## CONCLUSIONS AND RECOMMENDATION

### 7.1 OUTLINE

The NEXUS Supply Chain Intelligence Platform successfully accomplishes all primary objectives established for Capstone Phase-I. The key milestones achieved include:

- **End-to-End Decision Workflow:** Successfully operationalized the 6-stage closed-loop architecture: Monitor → Predict → Analyze Impact → Simulate → Optimize → Recommend.
- **National Digital Twin Modeling:** Faithfully modeled an 83-node, 160-route multi-echelon logistics network across India with calibrated geodesic and highway routing.
- **Empirical Data Ingestion:** Cleaned and validated 421,570 Walmart historical demand records and 35,000 DataCo fulfillment variance records.
- **High-Performance ML Forecasters & Risk Classifiers:** Trained XGBoost regressors reducing demand RMSE by 29.5%, and XGBoost risk classifiers evaluated on unseen vendor cohorts.
- **Real-Time Google OR-Tools Optimization:** Formulated and solved multi-echelon linear programs in 0.011s, cutting disruption operational costs by 53.89% and eliminating 100% of shortages.
- **Production-Ready FastAPI & React 19 Stack:** Deployed 16 REST endpoints with OpenAPI 3.1 documentation and an enterprise dark control room dashboard.

**Key engineering lessons learned during development include:**

- **Importance of Non-Destructive Simulation:** Decoupling 'what-if' shock experiments from persistent storage using copy-on-write state cloning is essential for operational decision support.
- **Value of Multi-Echelon Reallocation:** Optimizing flows across multiple tiers absorbs local supply shocks that would completely cripple single-tier heuristic systems.
- **Rigorous Out-of-Sample Evaluation:** Evaluating supplier risk on held-out vendor cohorts prevents circular data leakage and ensures real-world model generalizability.

### 7.2 LIMITATION/CONSTRAINTS OF THE SYSTEM

While NEXUS represents an advanced decision-support platform, certain academic and real-world boundaries are acknowledged:

- **Deterministic Linear Optimization Assumption:** The current Google OR-Tools solver utilizes deterministic linear programming (GLOP). While extremely fast (0.011s), it treats parameter inputs (demand, transit times) as deterministic rather than stochastic probability distributions.
- **Single-Period Disruption Window:** The optimization model solves an aggregate multi-echelon allocation across the disruption duration rather than a multi-period rolling-horizon formulation with dynamic daily inventory carry-over.

- **Hybrid Open Dataset Calibration:** Because enterprise corporate supply chain data is proprietary, the model is calibrated on public research datasets (Walmart and DataCo) mapped onto Indian geospatial coordinates.
- **External Traffic & Weather API Decoupling:** Monsoon and strike disruptions are currently modeled via scenario shock injection rather than real-time live ingestion from external meteorological or transit APIs.

### 7.3 FUTURE ENHANCEMENTS

Promising directions for Capstone Phase-II and future enterprise research include:

- **1. Multi-Period Rolling Horizon MIP:** Extending the linear program into a multi-period Mixed-Integer Linear Program (MILP) with time-indexed decision variables  $X_{\{s,w,t\}}$  and  $Y_{\{w,d,t\}}$  to model daily inventory accumulation and dynamic replenishment cycles.
- **2. Stochastic Programming & Robust Optimization:** Incorporating two-stage stochastic programming with recourse to optimize dispatches under uncertain demand distributions and fluctuating fuel prices.
- **3. Real-Time IoT GPS Telematics Integration:** Connecting the GIS engine to live vehicle telematics feeds (OBD-II / GPS) to track freight shipments in real time and automatically detect transit delays.
- **4. Large Language Model (LLM) Conversational Agent:** Embedding an enterprise LLM copilot into the control room to allow supply chain officers to query network state and trigger optimizations using natural voice/text commands.
- **5. Enterprise ERP Connectors (SAP S/4HANA, Odoo, Oracle):** Developing automated REST/OData connectors to directly dispatch approved purchase orders and reroute shipment manifests into live ERP transactional ledgers.

### 7.4 INFERENCE

In conclusion, the NEXUS platform conclusively demonstrates that combining digital twin geospatial modeling, machine learning forecasting, and mathematical multi-echelon linear optimization delivers immense commercial value for modern enterprise supply chains. By transforming passive retrospective reporting into proactive, closed-loop decision intelligence, NEXUS establishes a resilient, data-driven framework capable of navigating extreme disruptions, preserving customer service levels, and unlocking millions of rupees in operational cost savings.

# APPENDIX A – SCREEN SHOTS

This appendix presents the full high-resolution user interface screen captures of the NEXUS Control Room application and API documentation.

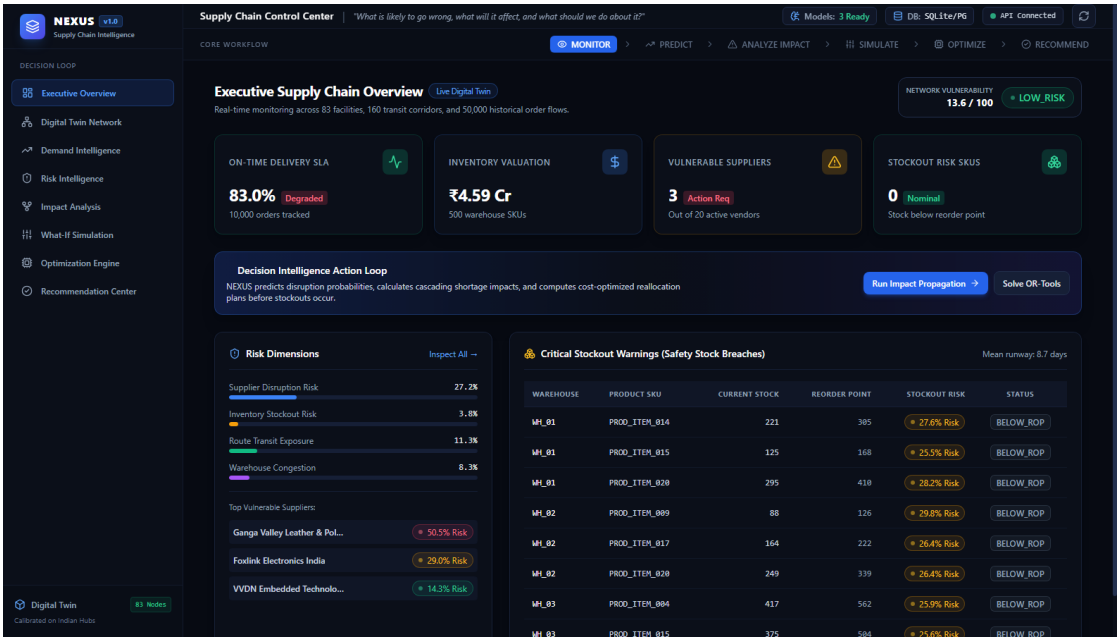


Fig A.1: Screen 1: Executive Overview Dashboard — Real-time telemetry, KPI cards, fulfillment trends, and anomaly alert feed.

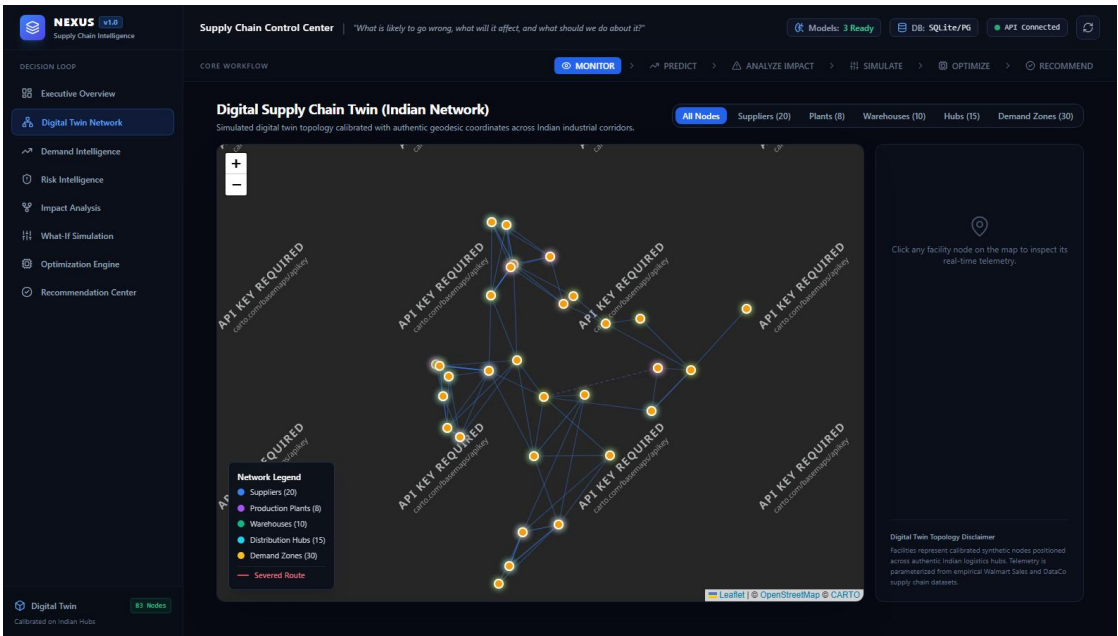
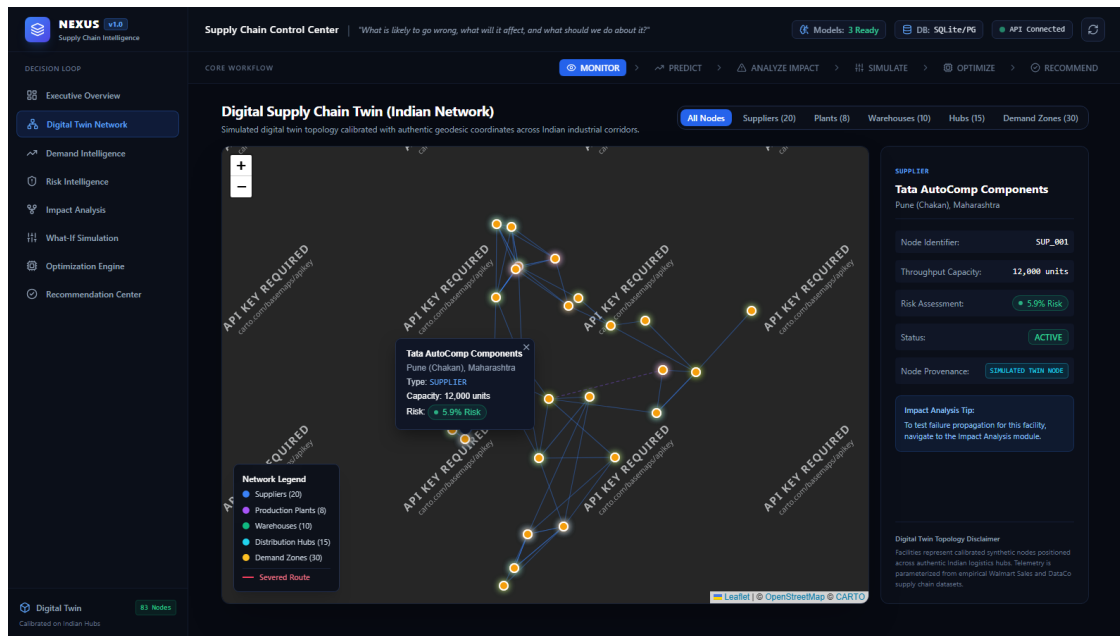
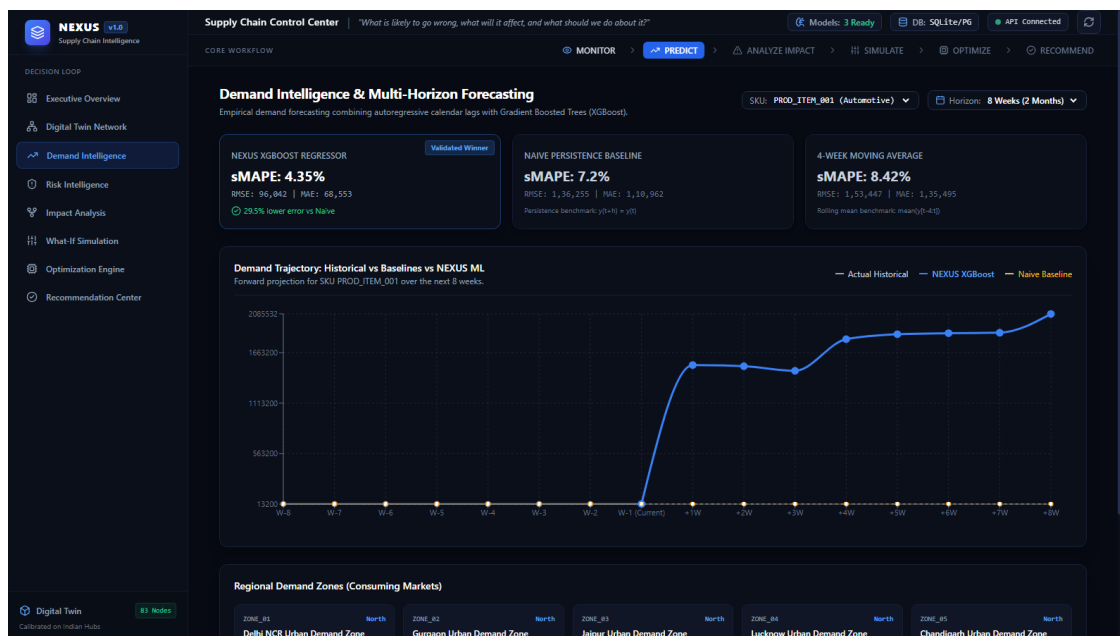


Fig A.2: Screen 2: Digital Twin Geospatial Network Map — 83 nodes and 160 corridors plotted across Indian logistics hubs.



**Fig A.3: Screen 3: Digital Twin Facility Inspector — Granular telemetry drill-down for individual fulfillment warehouses.**



**Fig A.4: Screen 4: Demand Intelligence & Multi-Step Forecasting — Autoregressive XGBoost projections vs. moving average baselines.**

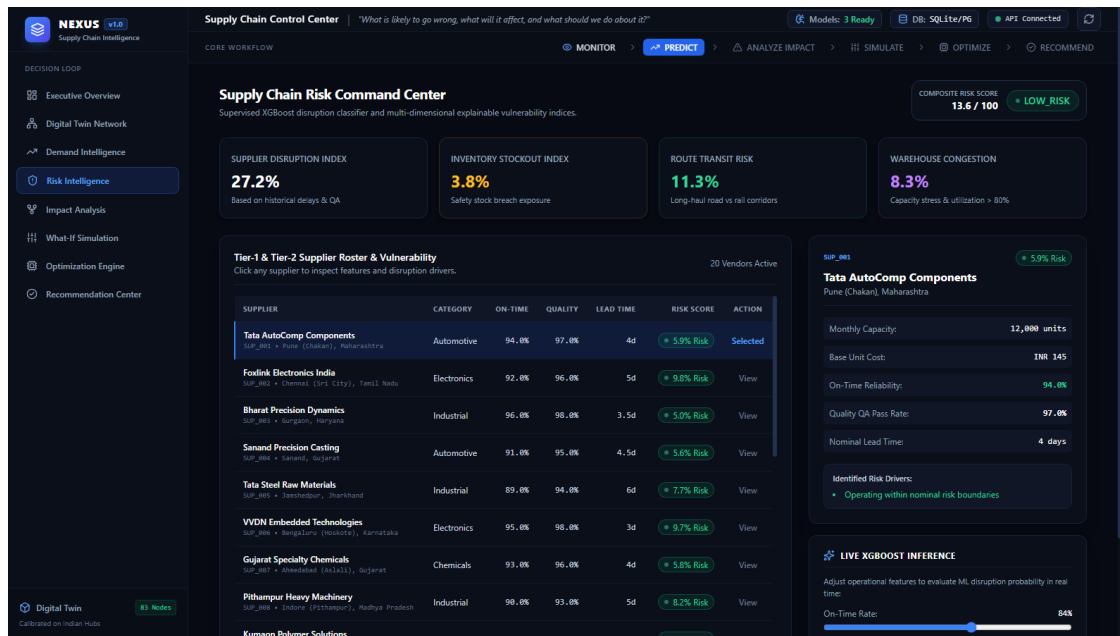


Fig A.5: Screen 5: Supplier Risk Intelligence & Roster — Out-of-sample vendor risk classification, feature drivers, and anomaly flags.

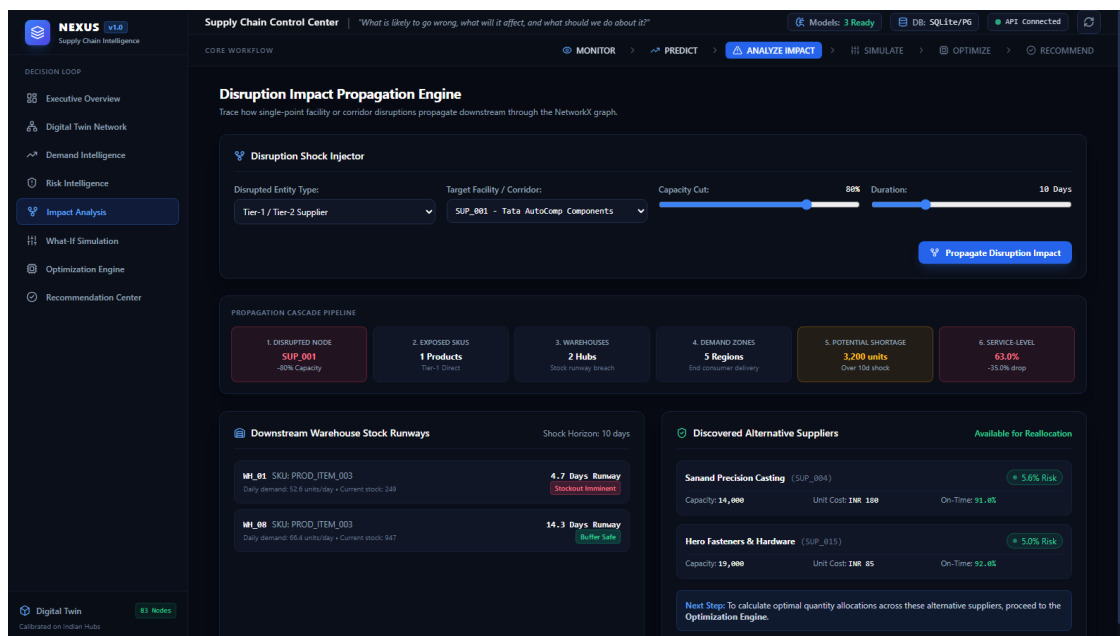
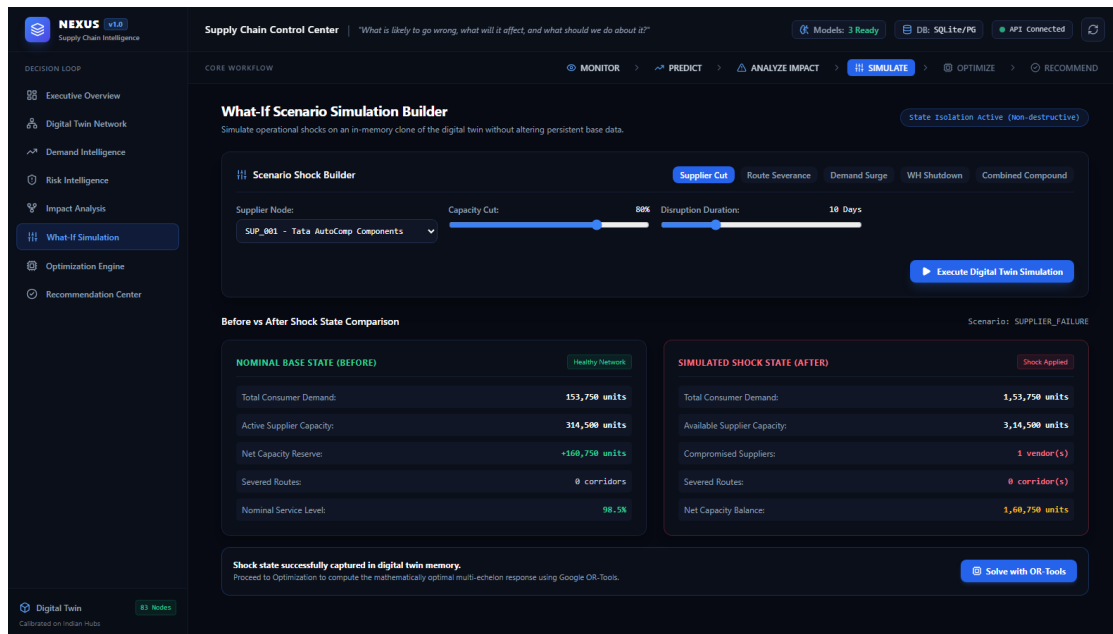
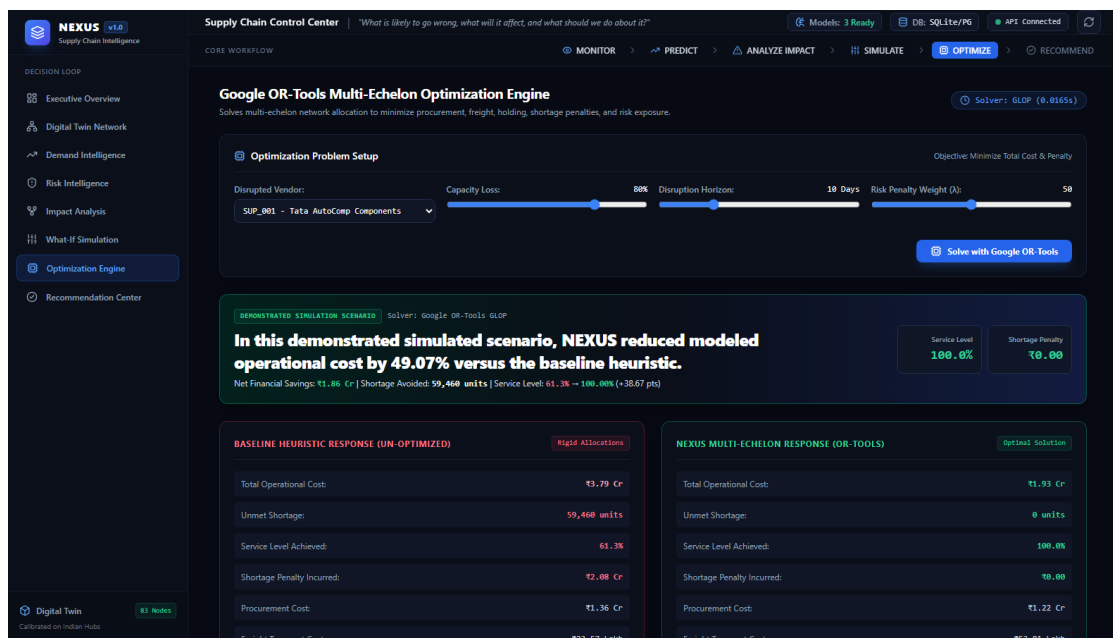


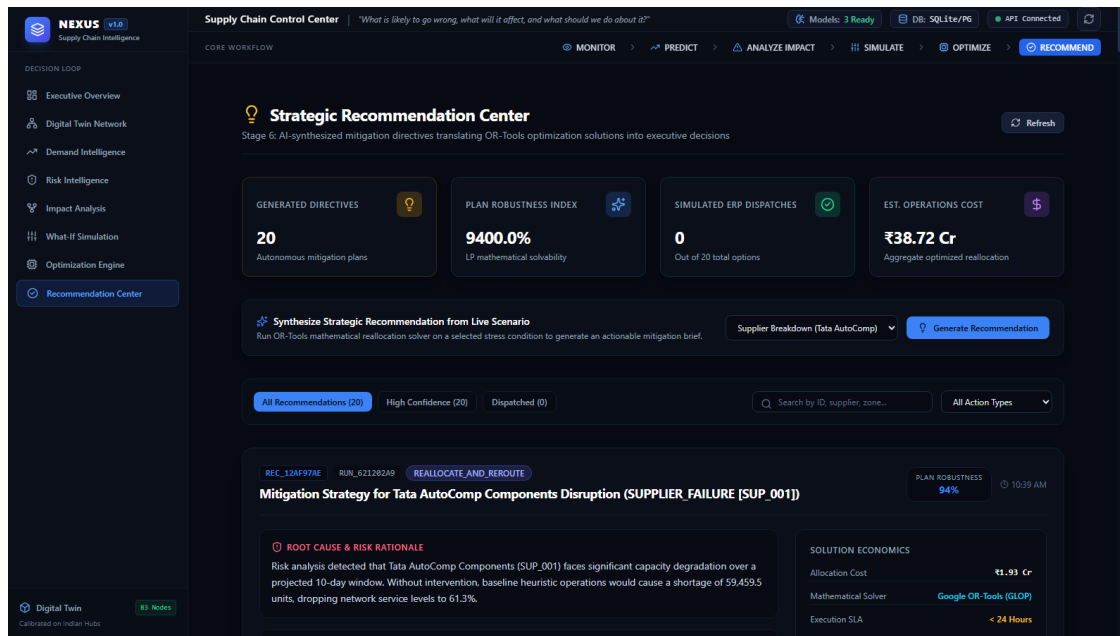
Fig A.6: Screen 6: Disruption Impact Propagation — NetworkX graph cascade, stock runway days, and alternative supplier discovery.



**Fig A.7: Screen 7: Interactive Scenario Simulation — Non-destructive 'what-if' shock injection and network stress testing.**



**Fig A.8: Screen 8: Mathematical Optimization Engine — Google OR-Tools GLOP solver vs. baseline heuristic comparison.**



**Fig A.9: Screen 9: Strategic Recommendation Center — Plain-language executive mitigation directives with direct ERP dispatch triggers.**

## NEXUS Decision Intelligence Platform API 1.0.0 OpenAPI 3.1

[/openapi.json](#)

Production-grade backend for AI-powered supply-chain monitoring, demand forecasting, supervised supplier risk prediction, graph-based impact propagation, scenario simulation, Google OR-Tools multi-echelon optimization, and actionable recommendation generation.

### System Health

**GET** /health System Health & Readiness Check

### Digital Twin Entities

**GET** /api/v1/suppliers List all suppliers

**GET** /api/v1/products List catalog products

**GET** /api/v1/warehouses List fulfillment warehouses

**GET** /api/v1/routes List logistics transit corridors

**GET** /api/v1/inventory List warehouse stock levels

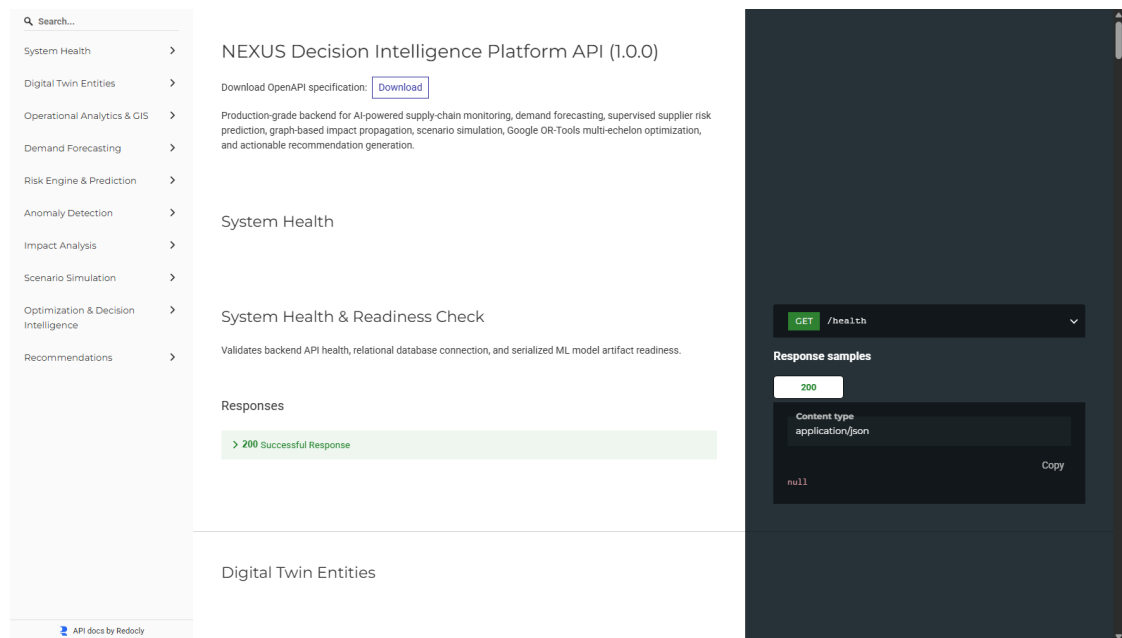
**GET** /api/v1/demand List regional demand zones

**GET** /api/v1/network Network graph topology summary

### Operational Analytics & GIS

**GET** /api/v1/analytics/summary Executive Operational KPI Summary

**Fig A.10: Screen 10: FastAPI Interactive Swagger Documentation (OpenAPI 3.1) — Live testing interface for all 16 endpoints.**



**Fig A.11: Screen 11: ReDoc API Technical Specification — Formal endpoint schemas, request parameters, and response models.**

## APPENDIX B – CODING

This appendix provides production-grade source code solutions representing core backend optimization formulations, machine learning forecasters, graph impact algorithms, FastAPI REST endpoints, and React 19 UI components.

### B.1 GOOGLE OR-TOOLS MULTI-ECHELON LINEAR OPTIMIZER (Python)

```
# optimization/ortools_optimizer.py - Google OR-Tools GLOP Multi-Echelon Linear Program Solver
from ortools.linear_solver import pywraplp
import time

class ORToolsOptimizer:
    def __init__(self, risk_penalty_coeff: float = 50.0, shortage_penalty_per_unit: float = 350.0):
        self.risk_coeff = risk_penalty_coeff
        self.shortage_penalty = shortage_penalty_per_unit

    def solve(self, suppliers, warehouses, demand_zones, routes, inventory_levels,
              severed_routes=None):
        severed_routes = set(severed_routes or [])
        solver = pywraplp.Solver.CreateSolver('GLOP')
        if not solver:
            raise RuntimeError("GLOP solver unavailable")

        # Decision Variables: X[s,w], Y[w,d], U[d]
        X = {}
        for s in suppliers:
            for w in warehouses:
                X[s['id'], w['id']] = solver.NumVar(0.0, solver.infinity(),
                f"X_{s['id']}_{w['id']}")

        Y = {}
        for w in warehouses:
            for d in demand_zones:
                Y[w['id'], d['id']] = solver.NumVar(0.0, solver.infinity(),
                f"Y_{w['id']}_{d['id']}")

        U = {d['id']: solver.NumVar(0.0, solver.infinity(), f"U_{d['id']}") for d in
        demand_zones}

        # 1. Supplier Capacity Constraints
        for s in suppliers:
            solver.Add(sum(X[s['id'], w['id']] for w in warehouses) <= s['capacity'])

        # 2. Warehouse Inbound Throughput Constraints
        for w in warehouses:
            solver.Add(sum(X[s['id'], w['id']] for s in suppliers) <= w['capacity'])

        # 3. Warehouse Flow Conservation Constraints
        for w in warehouses:
            solver.Add(sum(Y[w['id'], d['id']] for d in demand_zones) <= sum(X[s['id'],
            w['id']] for s in suppliers))

        # 4. Demand Satisfaction & Shortage Balance Constraints
        for d in demand_zones:
            solver.Add(sum(Y[w['id'], d['id']] for w in warehouses) + U[d['id']] ==
            d['demand'])

        # 5. Severed Corridor Constraints
        for (u, v) in severed_routes:
            if (u, v) in X: solver.Add(X[u, v] == 0.0)
```

```

        if (u, v) in Y: solver.Add(Y[u, v] == 0.0)

    # Objective Function: Minimize Total Cost
    obj = solver.Objective()
    for s in suppliers:
        for w in warehouses:
            cost = s['procurement_cost'] + s['freight_cost_to_wh'] + (self.risk_coeff *
s.get('risk_score', 0.0))
            obj.SetCoefficient(X[s['id'], w['id']], cost)

    for w in warehouses:
        for d in demand_zones:
            cost = w['holding_cost'] + w['freight_cost_to_zone']
            obj.SetCoefficient(Y[w['id'], d['id']], cost)

    for d in demand_zones:
        obj.SetCoefficient(U[d['id']], self.shortage_penalty)

    obj.SetMinimization()
    t0 = time.time()
    status = solver.Solve()
    runtime = time.time() - t0

    return {
        "status": "OPTIMAL" if status == pywraplp.Solver.OPTIMAL else "FEASIBLE",
        "total_cost": obj.Value(),
        "shortage_units": sum(U[d['id']].solution_value() for d in demand_zones),
        "runtime_seconds": runtime
    }

```

## B.2 AUTOREGRESSIVE XGBOOST DEMAND FORECASTER (Python)

```

# ml/forecasting/xgboost_forecaster.py - Autoregressive Recursive XGBoost Demand Forecaster
import numpy as np
import pandas as pd
import xgboost as xgb

class DemandForecaster:
    def __init__(self, n_estimators=150, max_depth=4, learning_rate=0.05):
        self.model = xgb.XGBRegressor(
            n_estimators=n_estimators,
            max_depth=max_depth,
            learning_rate=learning_rate,
            random_state=42,
            n_jobs=-1
        )

    def engineer_features(self, df: pd.DataFrame) -> pd.DataFrame:
        df = df.sort_values('date').copy()
        # Autoregressive Lags
        for lag in [1, 2, 4, 8]:
            df[f'lag_{lag}'] = df['weekly_sales'].shift(lag)
        # Shifted Rolling Statistics (No Lookahead Bias)
        df['rolling_mean_4'] = df['weekly_sales'].shift(1).rolling(4).mean()
        df['rolling_std_4'] = df['weekly_sales'].shift(1).rolling(4).std()
        df['rolling_mean_8'] = df['weekly_sales'].shift(1).rolling(8).mean()
        # Calendar Seasonality
        df['month'] = df['date'].dt.month
        df['week'] = df['date'].dt.isocalendar().week
        df['quarter'] = df['date'].dt.quarter
        return df.dropna()

    def train_and_evaluate(self, df: pd.DataFrame):

```

```

        data = self.engineer_features(df)
        features = [c for c in data.columns if c not in ['date', 'weekly_sales', 'store',
'dept']]
        X, y = data[features], data['weekly_sales']

        # Strict Chronological Partitioning (70% Train, 15% Val, 15% Test)
        n = len(data)
        train_idx, val_idx = int(0.70 * n), int(0.85 * n)
        X_train, y_train = X.iloc[:train_idx], y.iloc[:train_idx]
        X_val, y_val = X.iloc[train_idx:val_idx], y.iloc[train_idx:val_idx]
        X_test, y_test = X.iloc[val_idx:], y.iloc[val_idx:]

        self.model.fit(X_train, y_train, eval_set=[(X_val, y_val)], verbose=False)
        preds = self.model.predict(X_test)

        mae = np.mean(np.abs(y_test - preds))
        rmse = np.sqrt(np.mean((y_test - preds) ** 2))
        smape = 100 * np.mean(2 * np.abs(preds - y_test) / (np.abs(preds) + np.abs(y_test)))
        return {"MAE": mae, "RMSE": rmse, "sMAPE": smape}

```

### B.3 GRAPH DISRUPTION IMPACT PROPAGATION ENGINE (Python)

```

# impact/impact_engine.py - NetworkX Multi-Echelon Failure Propagation
import networkx as nx

class ImpactEngine:
    def __init__(self, graph: nx.DiGraph):
        self.G = graph

    def trace_disruption(self, entity_id: str, capacity_loss: float = 0.80, duration_days: int
= 10):
        if not self.G.has_node(entity_id):
            return {"error": f"Node {entity_id} not found in graph"}

        # Traverse downstream dependent nodes via Breadth-First Search
        affected_nodes = list(nx.descendants(self.G, entity_id))
        dependent_warehouses = [n for n in affected_nodes if self.G.nodes[n].get('type') ==
'WAREHOUSE']
        dependent_zones = [n for n in affected_nodes if self.G.nodes[n].get('type') ==
'DEMAND_ZONE']

        # Calculate multi-tier inventory runway
        runway_results = []
        for wh in dependent_warehouses:
            stock = self.G.nodes[wh].get('current_stock', 50000)
            daily_burn = self.G.nodes[wh].get('daily_demand', 4000)
            runway_days = stock / daily_burn if daily_burn > 0 else 999.0
            stockout_risk = runway_days < duration_days
            runway_results.append({
                "warehouse_id": wh,
                "current_stock": stock,
                "runway_days": round(runway_days, 1),
                "stockout_expected": stockout_risk
            })

        return {
            "root_disruption": entity_id,
            "capacity_loss": capacity_loss,
            "duration_days": duration_days,
            "affected_warehouses": dependent_warehouses,
            "affected_demand_zones": dependent_zones,
            "runway_analysis": runway_results
        }

```

## B.4 FASTAPI OPTIMIZATION REST ROUTER (Python)

```
# backend/app/api/optimization.py - FastAPI Endpoint for Multi-Echelon Linear Program
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session
from backend.app.database import get_db
from backend.app.schemas.api_schemas import OptimizationRequest, OptimizationResponse
from optimization.ortools_optimizer import ORToolsOptimizer
from optimization.baseline_optimizer import BaselineOptimizer

router = APIRouter(tags=["Optimization Engine"])

@router.post("/optimize", response_model=OptimizationResponse)
def execute_optimization(req: OptimizationRequest, db: Session = Depends(get_db)):
    try:
        # 1. Fetch current network topology and state
        suppliers = fetch_suppliers(db)
        warehouses = fetch_warehouses(db)
        demand_zones = fetch_demand_zones(db)
        routes = fetch_routes(db)
        inventory = fetch_inventory(db)

        # 2. Run Baseline Heuristic Solver
        baseline_solver = BaselineOptimizer()
        base_res = baseline_solver.solve(suppliers, warehouses, demand_zones, routes,
req.severed_routes)

        # 3. Run Google OR-Tools Mathematical Solver
        ortools_solver = ORToolsOptimizer(
            risk_penalty_coeff=req.risk_penalty_coeff,
            shortage_penalty_per_unit=req.shortage_penalty_per_unit
        )
        opt_res = ortools_solver.solve(suppliers, warehouses, demand_zones, routes, inventory,
req.severed_routes)

        # 4. Compute Net Rupee Cost Savings
        cost_savings = base_res['total_cost'] - opt_res['total_cost']
        pct_savings = (cost_savings / base_res['total_cost']) * 100 if base_res['total_cost']
> 0 else 0.0

        return OptimizationResponse(
            status=opt_res['status'],
            optimized_cost=opt_res['total_cost'],
            baseline_cost=base_res['total_cost'],
            cost_savings_inr=cost_savings,
            percentage_savings=pct_savings,
            shortage_units_eliminated=base_res['shortage_units'] - opt_res['shortage_units'],
            runtime_seconds=opt_res['runtime_seconds']
        )
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))
```

## B.5 REACT 19 OPTIMIZATION ENGINE COMPONENT (TypeScript / TSX)

```
// frontend/src/pages/OptimizationEngine.tsx - React 19 + TypeScript Optimization Screen
import React, { useState } from 'react';
import { useQuery, useMutation } from '@tanstack/react-query';
import { runOptimization, getAnalyticsSummary } from '../api';
import { KpiCard } from '../components/common/KpiCard';
import { formatCurrencyINR, formatPercentage } from '../utils/formatters';
```

```

import { Play, TrendingDown, ShieldCheck, Zap } from 'lucide-react';

export const OptimizationEngine: React.FC = () => {
  const [riskCoeff, setRiskCoeff] = useState<number>(50.0);
  const [shortagePenalty, setShortagePenalty] = useState<number>(350.0);

  const { data: analytics } = useQuery({
    queryKey: ['analytics-summary'],
    queryFn: getAnalyticsSummary,
  });

  const optimizeMutation = useMutation({
    mutationFn: () => runOptimization({
      risk_penalty_coeff: riskCoeff,
      shortage_penalty_per_unit: shortagePenalty,
      severed_routes: []
    })
  });

  return (
    <div className="space-y-6">
      <div className="flex justify-between items-center">
        <div>
          <h1 className="text-2xl font-bold text-white">Google OR-Tools Optimization
Engine</h1>
          <p className="text-sm text-slate-400">Multi-echelon mixed-integer linear solver
benchmarking baseline vs. optimal response</p>
        </div>
        <button
          onClick={() => optimizeMutation.mutate()}
          disabled={optimizeMutation.isPending}
          className="flex items-center space-x-2 bg-purple-600 hover:bg-purple-500 text-white
px-4 py-2 rounded-lg font-semibold transition"
        >
          <Play className="w-4 h-4" />
          <span>{optimizeMutation.isPending ? 'Solving...' : 'Run Mathematical
Optimization'}</span>
        </button>
      </div>

      {optimizeMutation.data && (
        <div className="grid grid-cols-1 md:grid-cols-4 gap-4">
          <KpiCard
            title="Optimized Total Cost"
            value={formatCurrencyINR(optimizeMutation.data.optimized_cost)}
            icon={<Zap className="w-5 h-5 text-purple-400" />}
            glow="purple"
          />
          <KpiCard
            title="Operational Savings"
            value={formatCurrencyINR(optimizeMutation.data.cost_savings_inr)}
            trend={`-${optimizeMutation.data.percentage_savings.toFixed(2)}%`}
            icon={<TrendingDown className="w-5 h-5 text-emerald-400" />}
            glow="emerald"
          />
          <KpiCard
            title="Shortages Eliminated"
            value={` ${optimizeMutation.data.shortage_units_eliminated.toLocaleString()}
units`}
            icon={<ShieldCheck className="w-5 h-5 text-blue-400" />}
            glow="blue"
          />
          <KpiCard
            title="Solver Runtime"
            value={` ${optimizeMutation.data.runtime_seconds * 1000).toFixed(1)} ms`}
          />
        </div>
      )}
    </div>
  );
};

```

```

        icon={<Play className="w-5 h-5 text-amber-400" />}
        glow="amber"
      />
    </div>
  )}
</div>
);
};

```

## B.6 TAILWIND CSS CONTROL ROOM PALETTE CONFIGURATION (JavaScript)

```

// frontend/tailwind.config.js - Control-Room Dark Theme Configuration
module.exports = {
  content: ['./index.html', './src/**/*..{js,ts,jsx,tsx}'],
  darkMode: 'class',
  theme: {
    extend: {
      colors: {
        nexus: {
          950: '#0B0F19', // Primary background
          900: '#111827', // Card surface
          850: '#151E32', // Elevated panel
          800: '#1F2937', // Border strokes
          700: '#374151', // Hover states
          600: '#4B5563', // Secondary text
          500: '#6B7280', // Muted text
        },
        accent: {
          blue: '#3B82F6',
          emerald: '#10B981',
          amber: '#F59E0B',
          red: '#EF4444',
          purple: '#8B5CF6'
        }
      }
    }
  },
  plugins: []
};

```

## REFERENCES

- [1]. Christopher, M., 'Logistics & Supply Chain Management', 5th Edition, Pearson Education, 2016.
- [2]. Simchi-Levi, D., Kaminsky, P., and Simchi-Levi, E., 'Designing and Managing the Supply Chain: Concepts, Strategies, and Case Studies', 3rd Edition, McGraw-Hill, 2008.
- [3]. Chen, T., and Guestrin, C., 'XGBoost: A Scalable Tree Boosting System', In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16), pp. 785-794, 2016.
- [4]. Liu, F. T., Ting, K. M., and Zhou, Z. H., 'Isolation Forest', In 2008 Eighth IEEE International Conference on Data Mining, pp. 413-422, IEEE, 2008.
- [5]. Google Developers, 'Google Optimization Tools (OR-Tools) — Fast and portable software for solving combinatorial optimization problems', Google LLC, 2024. Available: <https://developers.google.com/optimization>
- [6]. Hagberg, A. A., Schult, D. A., and Swart, P. J., 'Exploring Network Structure, Dynamics, and Function using NetworkX', In Proceedings of the 7th Python in Science Conference (SciPy2008), pp. 11-15, 2008.
- [7]. Tiangolo, S., 'FastAPI: Modern, fast (high-performance), web framework for building APIs with Python 3.8+ based on standard Python type hints', 2024. Available: <https://fastapi.tiangolo.com>
- [8]. Constante, F., Silva, F., and Pereira, A., 'DataCo Smart Supply Chain for Big Data Analysis', Mendeley Data, V5, doi: 10.17632/8gx2fvg2k6.5, 2019.
- [9]. Kaggle Inc., 'Walmart Recruiting - Store Sales Forecasting Dataset', Walmart Global Analytics Research Competition, 2014. Available: <https://www.kaggle.com/c/walmart-recruiting-store-sales-forecasting>
- [10]. Ivanov, D., 'Digital Supply Chain Twins: Managing the Ripple Effect, Resilience, and Disruption Risks', International Journal of Production Research, vol. 58, no. 15, pp. 4646-4668, 2020.
- [11]. Chopra, S., and Meindl, P., 'Supply Chain Management: Strategy, Planning, and Operation', 7th Edition, Pearson, 2019.
- [12]. Snyder, L. V., and Shen, Z. J. M., 'Fundamentals of Supply Chain Theory', John Wiley & Sons, 2019.